

Certen — Founder's Guide to the Whole System

Status note (read me): Certen is on **testnets** today (Ethereum Sepolia, Solana Devnet, Cardano Preview, and others). All blockchain addresses in these docs are testnet addresses and they **change as we upgrade contracts**.

1. The one-paragraph version

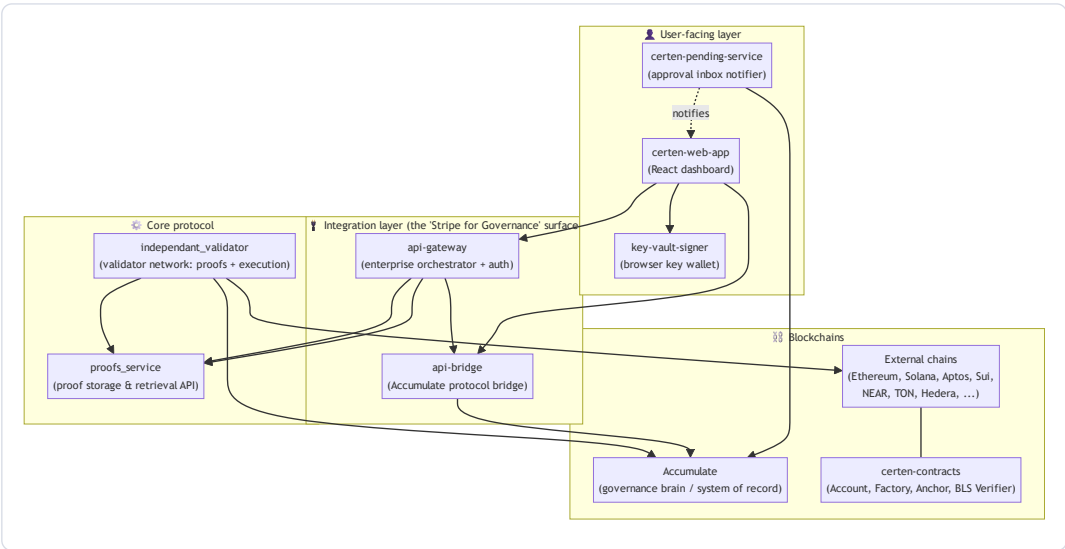
Certen lets an organization make a decision **once**, in one trusted place (the **Accumulate** blockchain, which acts as the company's "governance brain" and system of record), and then have that decision **carried out and proven** across many other blockchains (Ethereum, Solana, Aptos, Sui, NEAR, TON, and more). A network of **Certen validator nodes** watches Accumulate, packages each approved decision into a tamper-proof **cryptographic proof**, executes the action on the target chain through **Certen smart contracts**, and writes a receipt back to Accumulate. The result: a multi-chain action that **anyone can independently verify happened correctly**, without trusting Certen, any single validator, or any middleman.

2. The mental model (analogy)

Think of Certen as a **certified, multi-country courier and notary service** for corporate decisions:

Real-world analogy	Certen component	What it is
The boardroom + the official minute book	Accumulate (governance brain & system of record)	Where decisions are authorized and permanently recorded
The decision / signed resolution	Intent	A transaction on Accumulate that says "do X on chain Y"
The bonded courier network	Independent Validators	Nodes that watch Accumulate, prove the decision, and execute it elsewhere
Certified-mail receipts anyone can check	Proofs	Cryptographic evidence the decision was real, authorized, and executed
Lockboxes in each country that only open with a valid receipt	Certen smart contracts (Account, Factory, Anchor, BLS Verifier)	On-chain code that executes the action only when shown a valid proof
The corporate banking app	Web App	Where users see identities, approvals, history, and proofs
The hardware key / wallet	Key Vault Signer	Browser extension that holds private keys and signs approvals
The "you have something to sign" inbox	Pending Service	Watches for approvals you owe and notifies you
The Stripe-style API for partners	API Gateway / API Bridge	How enterprises and the app talk to the whole system

3. The eight repositories at a glance



Repo	Plain-English job	Tech
independant_validator	The heart of the network. Validator nodes that watch Accumulate, build proofs, run consensus among themselves, execute actions on external chains, and write results back.	Go, CometBFT
proofs_service	The library/archive. Stores every proof and serves it over an API so customers and auditors can fetch and verify them.	Go, PostgreSQL, React explorer
certen-contracts	The on-chain code deployed to every external chain: smart Accounts , the Factory that creates them, the Anchor that records & checks proofs, and the BLS Verifier that checks validator signatures cheaply.	Solidity (EVM) + Move/Rust/etc. for non-EVM
api-bridge	Low-level translator between a normal REST API and the Accumulate blockchain. Also deploys accounts on the 13 chains.	Node.js / Express
api-gateway	The enterprise front door. Multi-tenant, API-key auth, webhooks, signing providers (AWS KMS, Vault...), audit logs. Orchestrates the other services.	Node.js / Fastify, PostgreSQL
certen-pending-service	Background watcher that figures out "who needs to sign what" and pushes that to users' inboxes.	Node.js, Firestore
certen-web-app	The dashboard users actually click around in: identities, approvals, chains, proofs.	React
key-vault-signer	A browser extension that securely stores private keys and signs approvals. Keys never leave the extension.	Chrome Extension (MV3), React

4. Where to go next

These documents cover the system end to end — the first six are *how it works*, the seventh is *how to think about and sell it*:

1. **01 — Infrastructure & Workflow Diagrams** Visual maps of the full system and how data flows.
2. **02 — External Chains & their relationship to Accumulate** Which chains, which addresses, and exactly how they connect to Accumulate.
3. **03 — End-to-End Workflows** Every major capability, walked through step by step.
4. **04 — Proof Types & their Parts** What a "proof" actually contains, layer by layer.
5. **05 — How Proofs Are Used (where & when)** The life of a proof from creation to verification.
6. **06 — The Smart Contracts Explained** Account, Account Factory, BLS/ZK Verifier, and Anchor — what each does and how.
7. **07 — Possible Framing, and Monetization Paths** The dev team's perspective on what we've built and a few directions it might point — input for the business side, not a plan.
8. **08 — The API Gateway (For Builders)** The Stripe-style API — one key and a handful of high-level commands that let you drive the whole platform with code instead of the web app.

5. Mini-glossary (keep this handy)

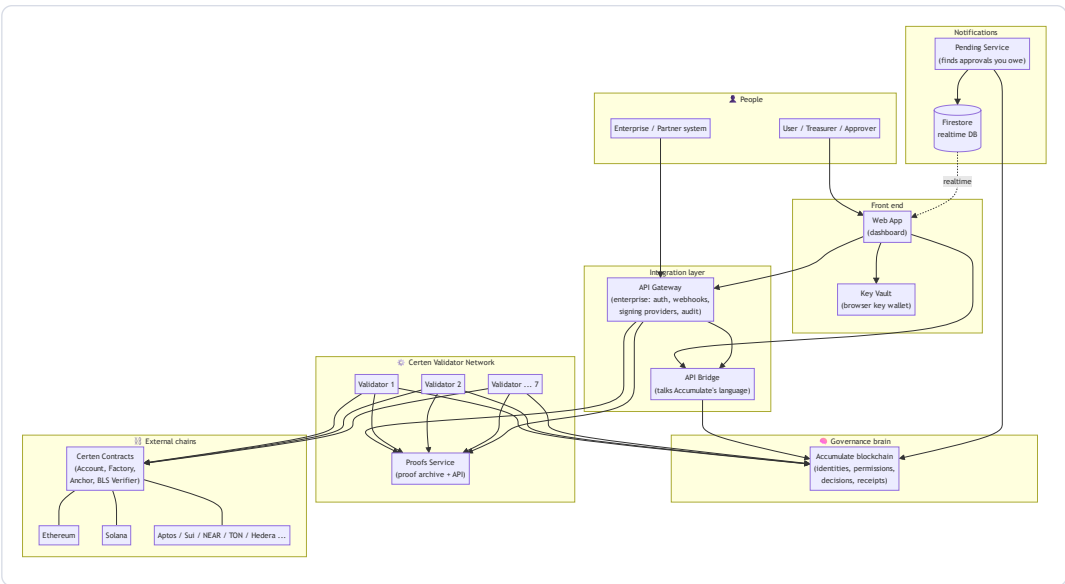
Term	Plain meaning
Accumulate	A blockchain purpose-built for identity & governance. Certen uses it as the trusted "decision book."
ADI (Accumulate Digital Identity)	A named on-chain identity, like <code>acc://acme.acme</code> . Think "the company's account name."
Key Book / Key Page	Accumulate's permission system. A key page lists who can sign and how many signatures are required (e.g. "3 of 5").
Intent	A request recorded on Accumulate to perform an action on another chain ("send 100 tokens on Ethereum").
Validator	A Certen node. The network needs a supermajority (2/3+) of honest validators to agree.
Consensus (BFT / CometBFT)	The voting process validators use to agree on the truth even if some are faulty or malicious.
Proof	Cryptographic evidence. Math, not trust — anyone can re-check it.
Anchor	(1) The act of recording a proof's fingerprint on an external chain; (2) the smart contract that does it.
app_hash / BPT root	Accumulate's one-number summary of all account states at a moment in time. A proof ties back to this.
Merkle proof	A compact way to prove "this item is inside this big set" using a chain of hashes.
BLS signature	A signature scheme that lets many validators' signatures be combined into one small signature — efficient to check on-chain.
ZK-SNARK / Groth16	A way to prove "I checked all those signatures and they're valid" with a tiny, cheap-to-verify proof.
G0 / G1 / G2	The three governance proof levels: was it included, was it authorized, did the outcome match.
L1–L4	The consensus/state proof layers: account state → block → validator signatures → genesis trust.
on-demand vs on-cadence	Two proof speeds: immediate (premium) vs batched every ~15 min (cheaper, amortized).

01 — Infrastructure & Workflow Diagrams

[← Back to index](#)

This document is the **picture book**. If you only read one page, read this one. Each diagram has a plain-English caption underneath.

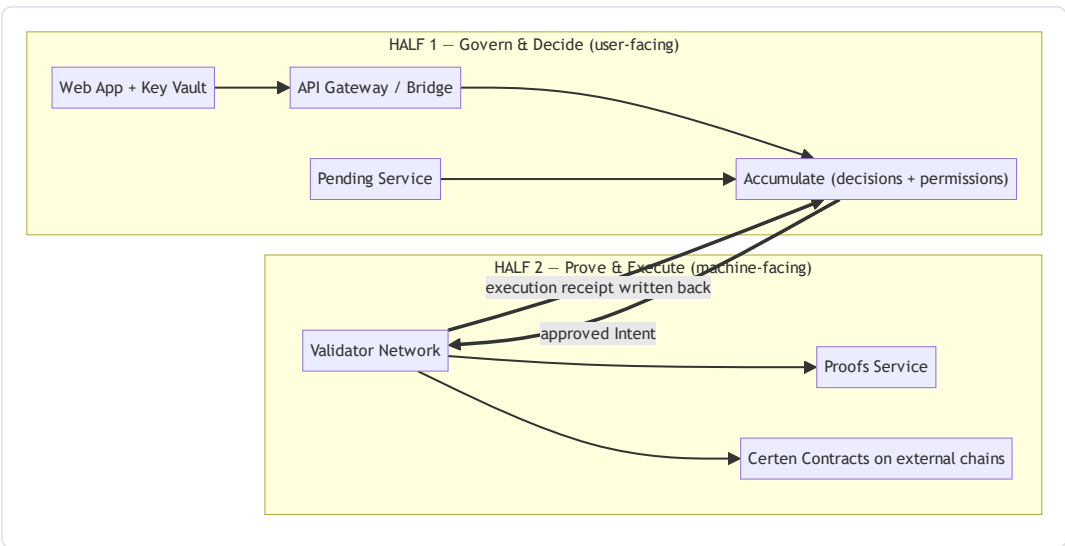
1. The whole system on one page



In words: People use the **Web App** (signing with the **Key Vault**). The app talks to Certen through the **API Gateway / Bridge**. Decisions land on **Accumulate**. The **Validator Network** watches Accumulate, proves each decision, and executes it on **external chains** via the **Certen Contracts**, storing every proof in the **Proofs Service**. The **Pending Service** keeps users informed about approvals they still owe.

2. The two "halves" of Certen

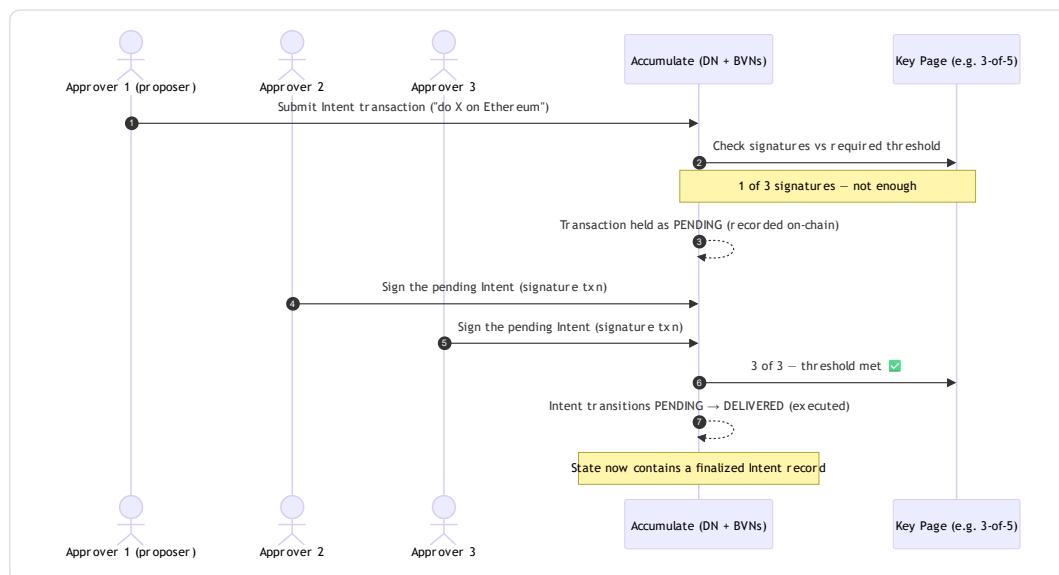
It helps to split Certen into two cooperating halves:



In words: Half 1 is about **deciding** (humans, identities, approvals) and lives around Accumulate. Half 2 is about **proving & doing** (validators, math, external chains). The bridge between them is the **Intent** going in and the **receipt** coming back.

3. Before the validators: how an Intent is born on Accumulate

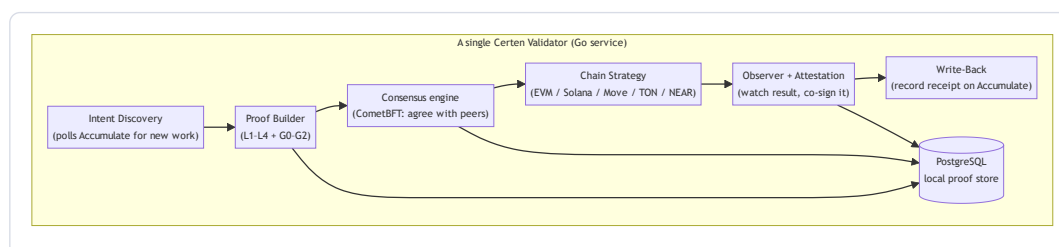
Everything in this guide *starts* with a finalized Intent sitting on Accumulate. But where does that Intent come from, and how does it get approved? This all happens on the **Accumulate** side, **before** any Certen validator is involved.



- 1. Writing the Intent.** An Intent is just a **transaction on Accumulate**, submitted to the organization's identity (its **ADI**, e.g. `acc://acme.acme`). It says, in structured form, "perform *this* action on *this* chain." Nothing leaves Accumulate yet — it's a request recorded in the governance brain.
- 2. Who is allowed to approve it (thresholds).** Every ADI is governed by a **Key Book**, and inside it a **Key Page** lists the authorized signing keys and an **accept threshold** — the classic "**M of N**" rule (e.g. 3 of 5 treasury officers). The threshold is the on-chain policy that says how many of the named authority signers must agree before anything is allowed to happen.
- 3. How multi-sig / pending transactions work.** If the first submission doesn't already carry enough signatures to meet the threshold, Accumulate does **not** reject it — it records the transaction in a **PENDING** state on-chain. Each additional approver then sends a small **signature transaction** that references the pending Intent's hash. Signatures accumulate over time (minutes, hours, or days — whatever the approvers take). This is exactly what the **Pending Service** later watches, so it can nudge "you still owe a signature."
- 4. How Accumulate nodes process all this.** Accumulate is itself a **BFT blockchain** (a Directory Network + shared BVNs). Its validators order and commit every one of these transactions — the original Intent and each signature — into blocks. Each committed block updates Accumulate's **app_hash / BPT root**, the one-number summary of all account state. So the pending Intent, and every signature attached to it, are **permanently recorded** as they arrive, not just at the end.
- 5. What happens when the threshold is met.** The moment the accumulated signatures from the authority signers reach the Key Page's threshold, Accumulate transitions the transaction from **PENDING** → **DELIVERED**: it executes and its effects are written into state. *Only now* does a **finalized, committed Intent record** exist on Accumulate. That finalized record — bound to a specific block and `app_hash`, approved by exactly the right key page — is precisely what a Certen validator discovers in the next step and what its proofs later bind to.

The handoff: Accumulate's job is to decide *correctly and provably*. By the time a Certen validator sees an Intent, the "who approved it, and were they allowed to" question is already settled and recorded on-chain. The validator network's job is to *prove that fact to other blockchains and carry out the action* — which is what the rest of this guide covers.

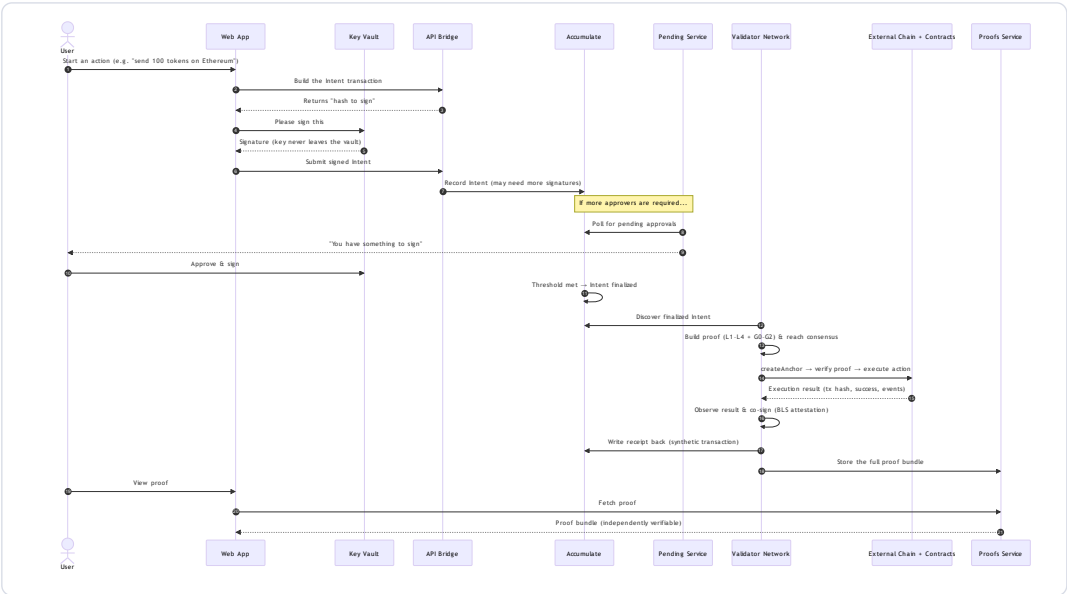
4. Inside one validator node



In words: Each node runs a small assembly line: **find** an approved intent → **build** its proof → **agree** with the other validators → **execute** it on the target chain → **watch and co-sign** the result → **write the receipt back** to Accumulate. Everything is logged in the node's own database.

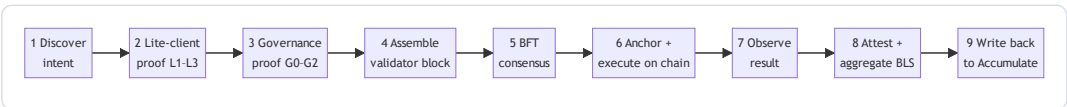
5. The end-to-end "happy path" (most important diagram)

This is the full life of one cross-chain action, from a person clicking *Approve* to an independently verifiable receipt.



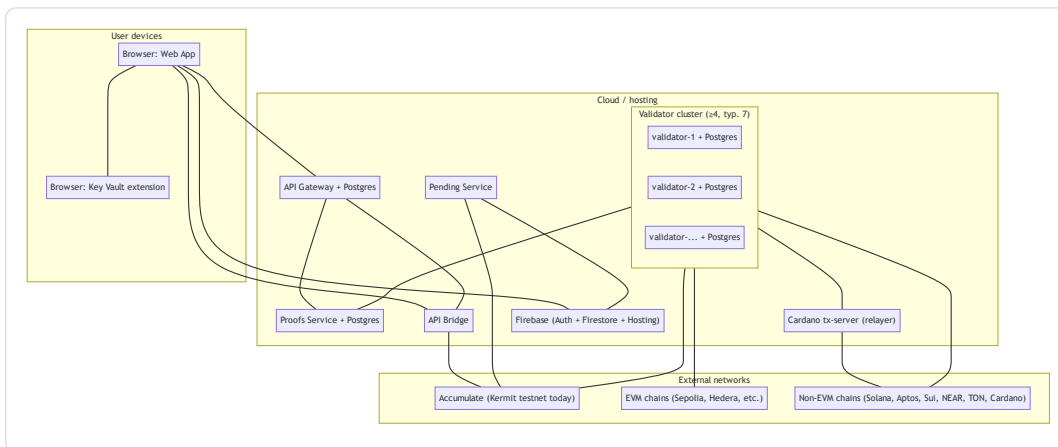
In words: A user authorizes an action; it gets the required approvals on Accumulate; the validator network proves and executes it on the target chain, co-signs the outcome, and records a receipt; the proof is archived and the user can pull it up and verify it.

6. The validator's 9-phase pipeline (detail of step "Build proof & execute")



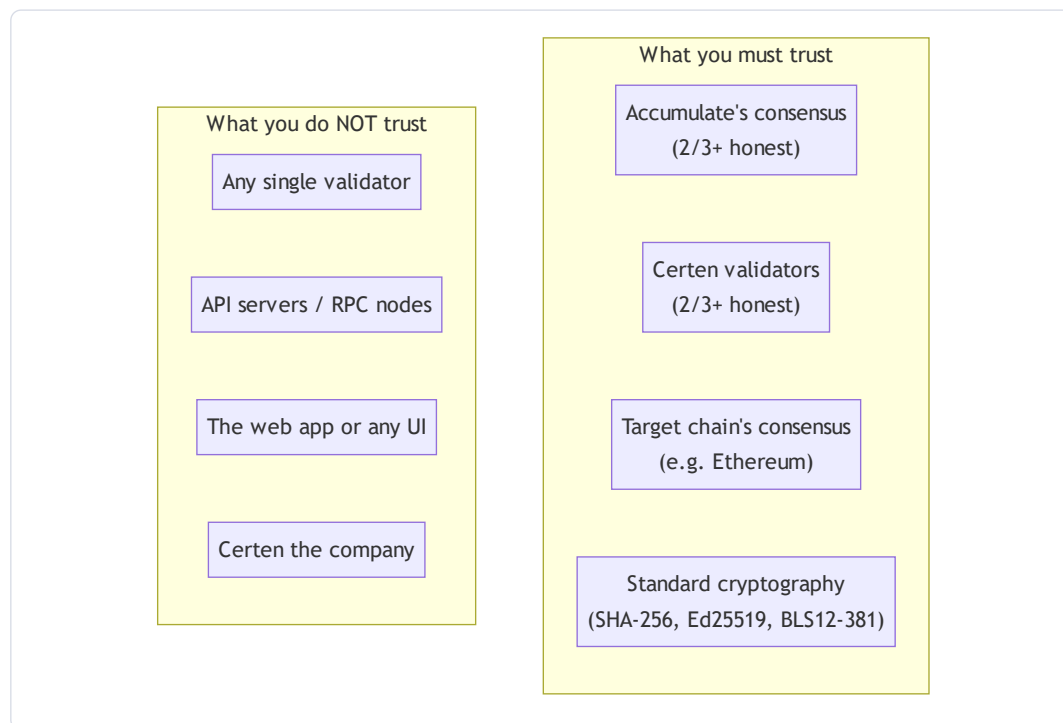
Phase	What happens	Why it matters
1 Discover	Node spots a finalized CERTEN_INTENT on Accumulate	The trigger for all work
2 Lite-client proof (L1-L3)	Prove the account/state is real and was committed by Accumulate consensus	Ties the action to Accumulate's truth
3 Governance proof (G0-G2)	Prove it was <i>included</i> , <i>authorized</i> , and the <i>outcome is bound</i>	Proves the right people approved it
4 Assemble	Pack all proof pieces into one canonical "validator block"	One object to agree on
5 Consensus	2/3+ of validators sign the block	No single validator can cheat
6 Anchor + execute	Call the on-chain contracts: record the anchor, verify the proof, perform the action	The action actually happens
7 Observe	Watch the target chain until the transaction is confirmed	Confirms it really executed
8 Attest + aggregate	Each validator co-signs the result; signatures combined via BLS + a ZK proof	A compact, verifiable group endorsement
9 Write-back	Record the result receipt back on Accumulate	Closes the loop; permanent audit trail

7. Deployment topology (where things physically run)



In words: The **validator cluster**, **proofs service**, **gateway**, **bridge**, and **pending service** run as backend services (each with its own database where needed). The **web app** is hosted on Firebase and runs in the user's browser alongside the **Key Vault** extension. Everything connects out to **Accumulate** and the **external chains**. Most chains are driven directly by the validators; **Cardano** is the exception — because of its UTXO/Plutus model the validators hand transactions to a small **Cardano tx-server (relayer)** that builds and submits them, then reports the result back.

8. Trust & fault tolerance (why you can believe it)



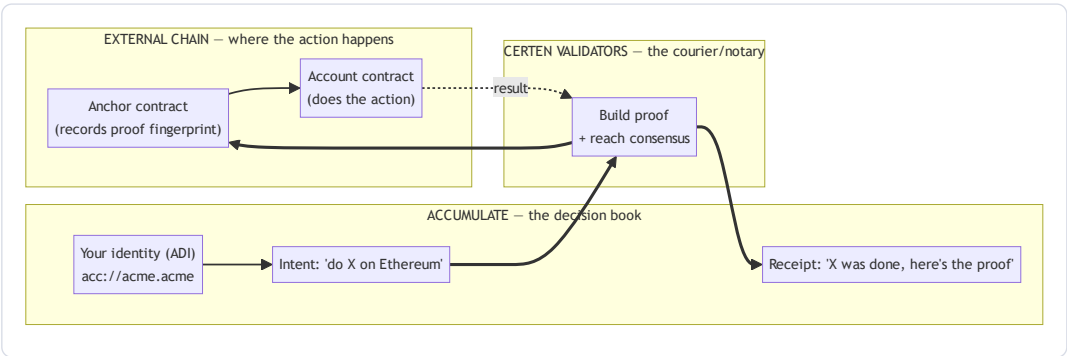
In words: Security rests on **math** and on **supermajorities**, not on any one company or server. If up to one-third of validators are malicious or broken, the system still produces only correct, verifiable results. Anyone can re-check a proof themselves — the UIs and servers are conveniences, not trust anchors.

02 — External Chains & Their Relationship to Accumulate

[← Back to index](#)

This document answers: **which blockchains does Certen touch, what are the contract addresses, and how exactly do those external chains connect back to Accumulate?**

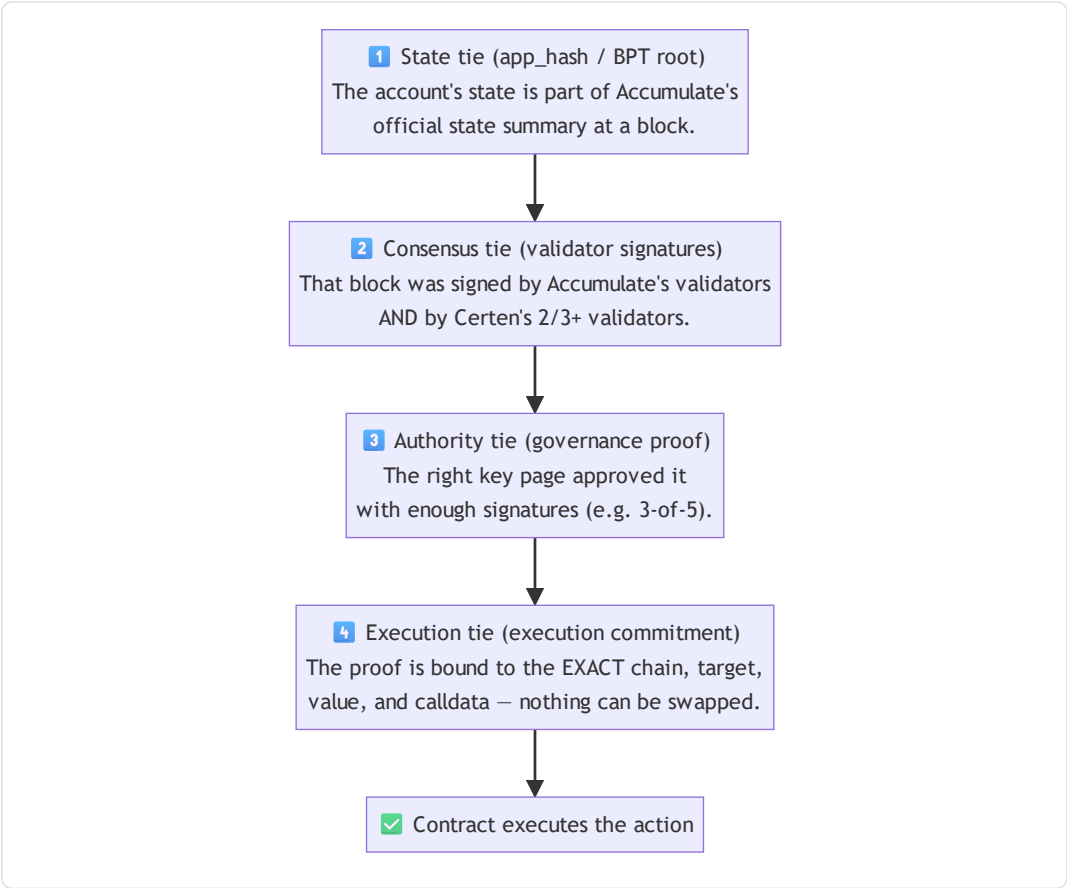
1. The core relationship in one picture



The key idea: Accumulate is the **single source of authority**. An external chain never decides anything on its own — it only acts when a Certen contract is shown a **proof that an Accumulate decision authorized this exact action**. The proof's fingerprint gets **anchored** on the external chain, and the result gets **written back** to Accumulate. The two worlds are stitched together by the validators and by the cryptographic proof.

2. How an Accumulate decision becomes an on-chain fact

There are four cryptographic "ties" that bind the two worlds together. Each one is checked on-chain before any action runs:



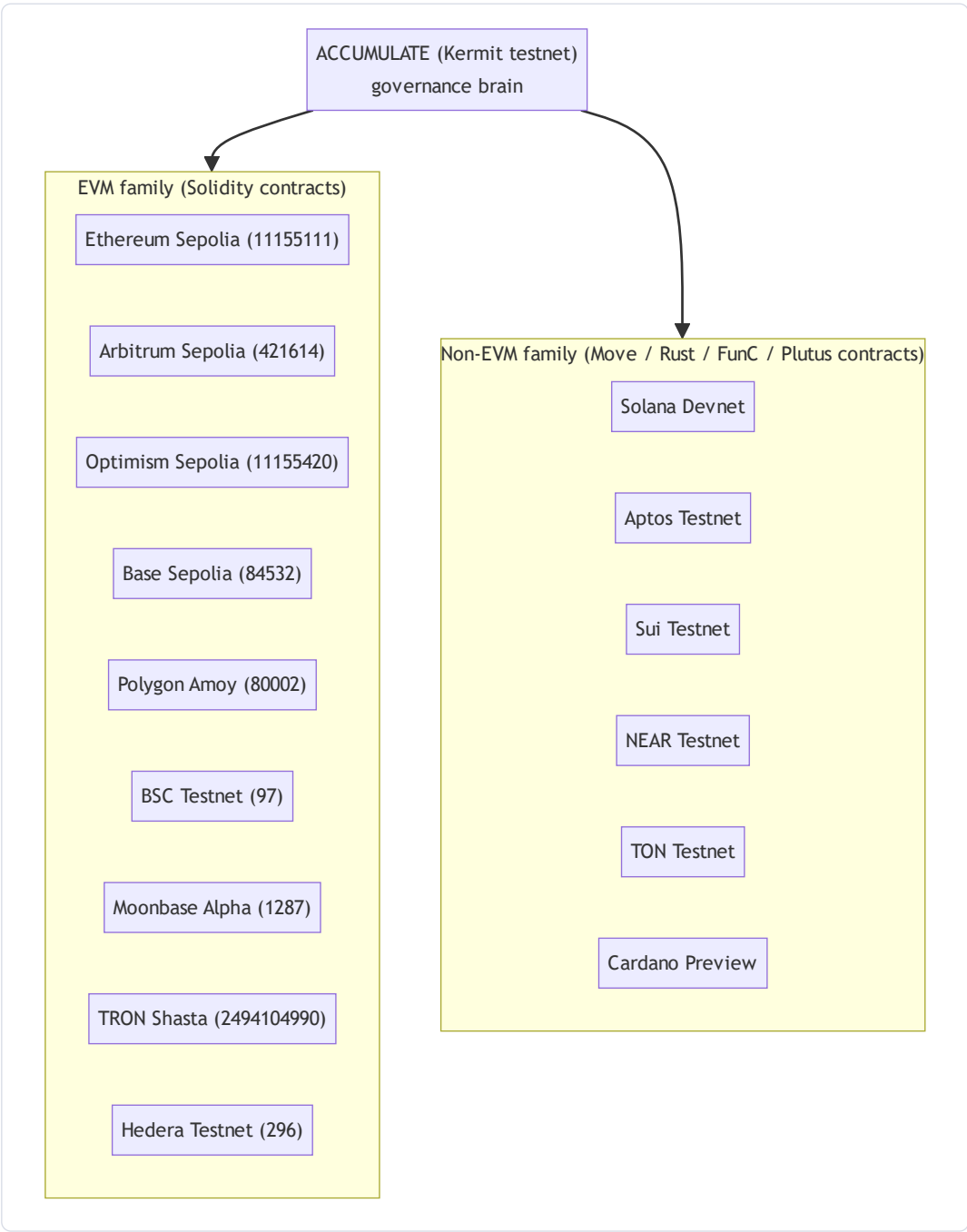
Plain English:

- **State tie** — "this account really existed in this state on Accumulate."
- **Consensus tie** — "Accumulate and Certen's validators both vouched for it."
- **Authority tie** — "the people who were allowed to approve it, did."
- **Execution tie** — "this approval is for *this* action on *this* chain only" (so a proof for a \$10 transfer can't be replayed as a \$1M transfer, or reused on a different chain).

The **execution tie** is one of Certen's most important security properties. Earlier contract versions could be tricked by swapping parameters; the current **V6.1** contracts bind the proof to six independent fields including the chain ID, the exact target/value/calldata, the intent's identity, and a snapshot of the validator set. See [06 — Contracts](#).

3. Which chains Certen supports

Certen runs against **Accumulate (Kermit testnet)** as the brain, and executes on **15 external testnets** today — a mix of EVM and non-EVM:



EVM chains all share the same Solidity contracts (Account, Factory, Anchor, BLS Verifier). **Non-EVM chains** have functionally-equivalent contracts written in each chain's native language (Move for Aptos/Sui, Rust for Solana/NEAR, FunC for TON, Plutus for Cardano). The validator talks to each family through a pluggable "chain strategy."

Cardano uses a helper relayer. Most chains are driven directly by the validators. Cardano's UTXO/Plutus model makes transactions awkward to build in-process, so Certen runs a small **Cardano tx-server** (a transaction-builder relayer) alongside the validators on the network's server. The validators tell it what to execute; it builds, submits, and reports back the result, which the validators then prove and write back to Accumulate like any other chain.

4. The current live addresses (testnet)

Every chain below runs the **current V6.1 ("A++") contracts** — the anchor with full security binding, the V6 account factory, and the V2 BLS verifier. There are no legacy (V3/V4/V5) contracts in the live path.

4a. Ethereum Sepolia (chain 11155111) — the primary EVM testnet

Contract	Address	Role
Anchor V6.1 (active)	0x14885Fe8b13605A01A9d62eB2dBf1bF3bB4ac368	Current anchor with full security binding
BLS Verifier V2 — adapter	0x8EEDa48f99709e90e30bE1510972b80163fd1aC7	Entry point used by Anchor V6.1
BLS Verifier V2 — generated	0x54c644A1ed515f3cb2Fa02c23F2ba09C2003aF22	The ZK (Groth16) verifier proper
Account Factory	0x9a8e3df1Da609CC1e7A3fA8df2B623266A6d5Da3	Creates user smart accounts

4b. Other EVM testnets (Anchor V6.1 + Account Factory)

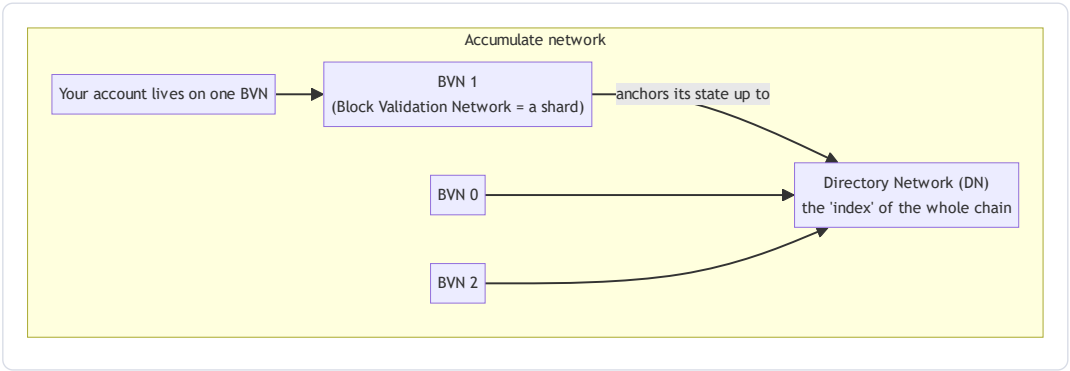
Chain	Anchor V6.1	Account Factory
Arbitrum Sepolia (421614)	0xad81055b211c2d99672A1b8586Dba4317F8896Dd	0x479ECeEb94AD4f68628a4C19c5126c77fD350cB6
Optimism Sepolia (11155420)	0xC2494b9945460fEb6e02397156B6C3e2bccB7c81	0x479ECeEb94AD4f68628a4C19c5126c77fD350cB6
Base Sepolia (84532)	0xbEAA8aA739d291782Fc90af9eD247e2e4430eade	0x479ECeEb94AD4f68628a4C19c5126c77fD350cB6
Polygon Amoy (80002)	0x7a8c5DC01C2d2Ba498F76832d8cbf0Fe2f69a6C3	0x9a8e3df1Da609CC1e7A3fA8df2B623266A6d5Da3
Moonbase Alpha (1287)	0x0f7E55FD33D9f20fe0bd18d7bdB5F2EA8dDDf812	0x479ECeEb94AD4f68628a4C19c5126c77fD350cB6
BSC Testnet (97)	0x7a8c5DC01C2d2Ba498F76832d8cbf0Fe2f69a6C3	0x479ECeEb94AD4f68628a4C19c5126c77fD350cB6
TRON Shasta (2494104990)	V6.1 (deploying)	(deploying)
Hedera Testnet (296)	0xca67409d872B04cc2a13dEAb2eaa2AD070029F59	0x964B43aF4C894CC29FE79C38f55Ca9889B6252D3

Hedera is fully upgraded to V6.1 with end-to-end value transfers verified. TRON Shasta is mid-upgrade to V6.1; its new addresses land this week.

4c. Non-EVM testnets

Chain	Anchor	Account Factory
Solana Devnet	program <code>Bf9cZdQwvGgynBaJYQYdywszZJQPR9TtZvu1ZNxR7JAU</code>	program <code>HYjh3ygUSFKd8EU8VrEbeKttCWSVxEsAT5zNqhBvS4BW</code>
Aptos Testnet	package <code>0x0d61023cca621d94f3b5a08421e2dff488897fd76691c53754d4ff457e5694d1</code>	(same package)
Sui Testnet	package <code>0x3914c9fc61bd7b78b96d93644641943296be530bc9cce51a436e5ead8fb20913</code>	object <code>0x36394f9d727832583bbfa131f2084ff5d7df736b6eef0b300c</code>
NEAR Testnet	<code>v6-1.certen-anchor-v5.testnet</code>	<code>v3.certen-factory-v2.testnet</code>
TON Testnet	<code>kQBgCKTdtxbMljgZT_Z1rD3k_gqmjZv7Q9tiPgGQEfjizAulz</code>	<code>kQBIGMx1eR_8S7j8gA1M_eSMw6As91hRAewZdxLthZLA5QnT</code>
Cardano Preview	<code>addr_test1wqqp0z9vxsus2940cfgd74d5027gtj9ta33y05z9vp1yqfq5y08y7</code>	(via Cardano tx-server relay)

5. How Accumulate itself is structured (just enough to follow proofs)



- Accumulate is **sharded**: accounts live on a **BVN** (Block Validation Network). Each BVN periodically **anchors** its state into the **Directory Network (DN)**, the master index.
- Each block carries an **app_hash** (a.k.a. **BPT root**) — a single number summarizing all account states.
- A Certen proof walks this ladder: *your account* → *BVN block (app_hash)* → *DN* → *validator signatures*. That's what the **L1–L4 layers** in the next docs mean.

Certen's validator config points at the Kermit testnet's DN and BVN endpoints (`ACCUMULATE_COMET_DN` , `ACCUMULATE_COMET_BVN...`) so it can read these blocks and their signatures directly.

6. The Certen network's own Accumulate identity

The validator network operates under its **own** Accumulate identity, used to write execution receipts back on-chain:

Item	Value
Network ADI	<code>acc://certen-protocol.acme</code>
Key book / page	<code>acc://certen-protocol.acme/book / ../book/1</code>
Receipts data account	<code>acc://certen-protocol.acme/execution-results</code>

When the validators finish executing an action (Phase 9), they write the result and its proof summary into the `execution-results` data account — a permanent, queryable audit trail on Accumulate itself.

03 — End-to-End Workflows (every capability, step by step)

[← Back to index](#)

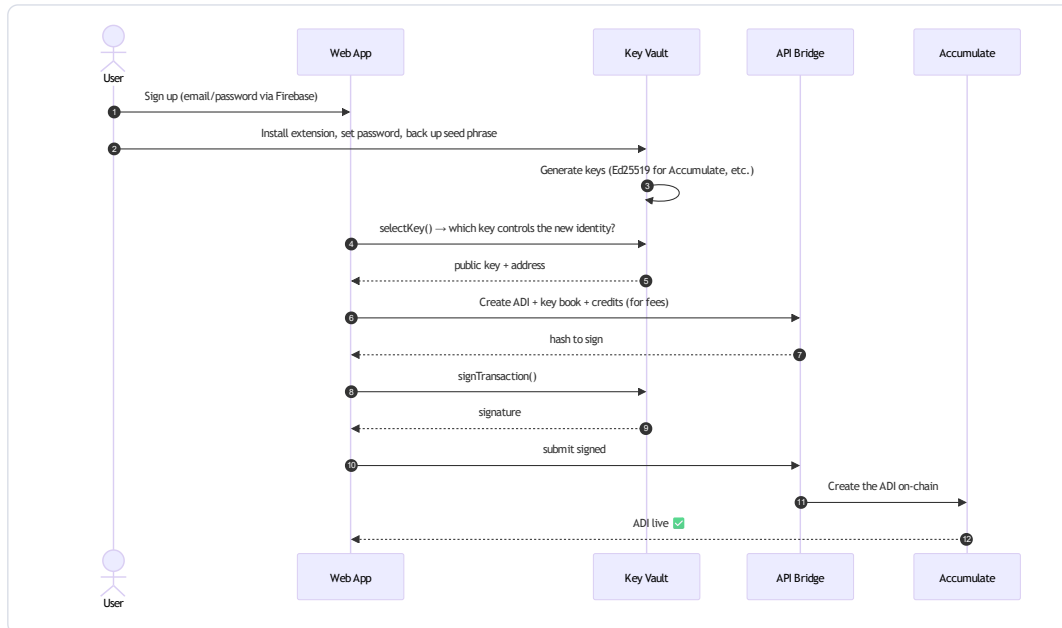
This document walks through **every major thing Certen can do**, from the user's first click to the final verifiable result. Each workflow has a plain-English summary, a diagram, and a numbered walkthrough.

Capabilities covered:

1. [Onboarding: create an identity \(ADI\)](#)
2. [Setting up governance \(key pages, thresholds, delegation\)](#)
3. [Linking external-chain accounts](#)
4. [A single cross-chain action](#)
5. [A multi-signature approval](#)
6. [A multi-leg cross-chain action](#)
7. [On-demand vs on-cadence proofs](#)
8. [Viewing & independently verifying a proof](#)
9. [Enterprise integration via the API Gateway \(the full lifecycle\)](#)

1. Onboarding — create an identity (ADI)

Plain English: A new user creates their on-chain identity (an ADI, like `acc://acme.acme`) and a key to control it. The key is generated and stored in the browser Key Vault; only signatures ever leave it.

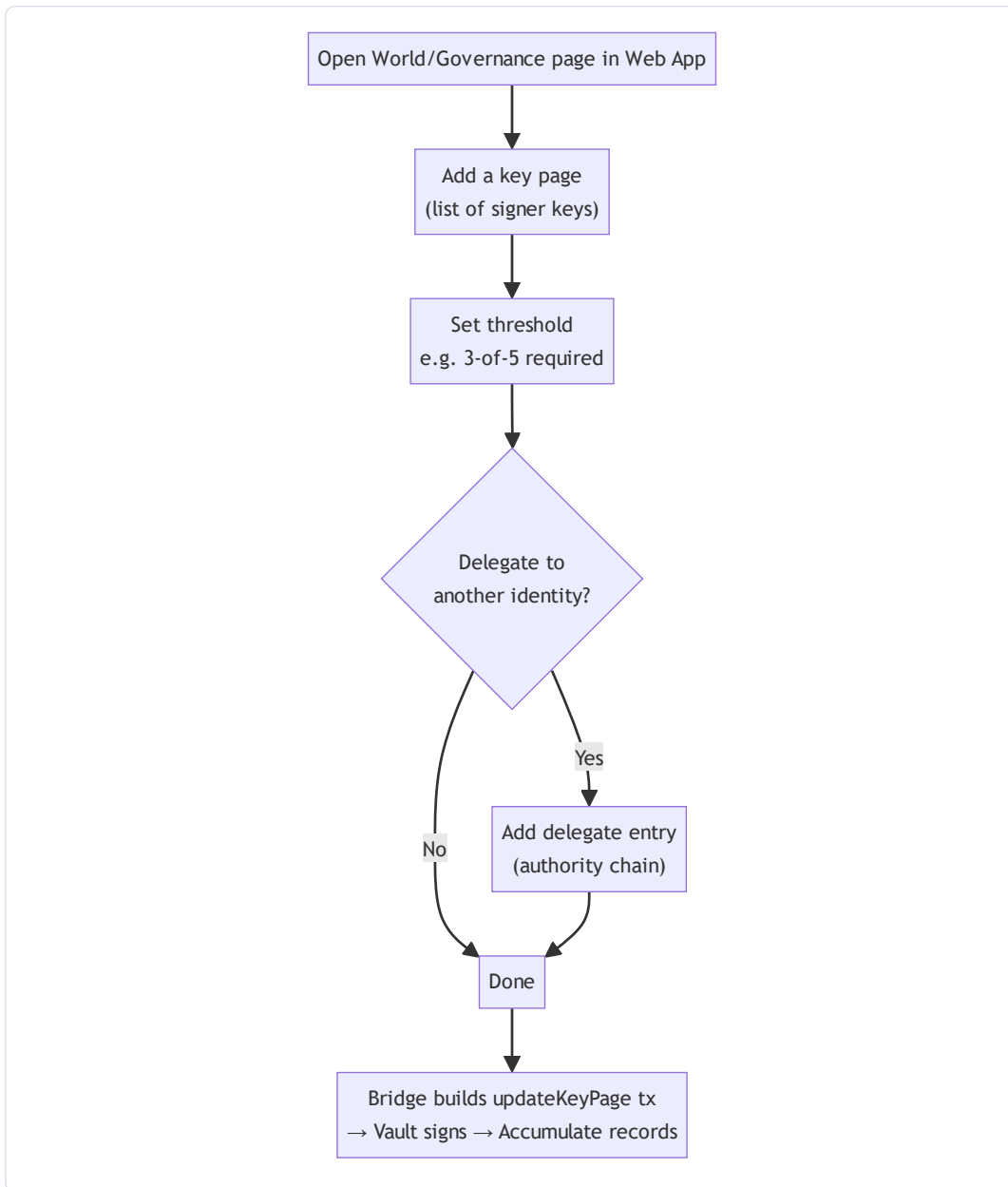


Walkthrough:

1. User signs up (Firebase Auth handles login).
2. User installs the **Key Vault** extension and creates a password; a 12/24-word seed phrase is generated and backed up. Keys are encrypted at rest.
3. The web app asks the Vault for a public key to base the identity on.
4. The **API Bridge** builds the Accumulate transactions that create the ADI, its key book (permissions), and buys "credits" (Accumulate's gas).
5. The Vault signs; the bridge submits to Accumulate. The identity is now live.

2. Setting up governance

Plain English: The user defines *who can approve what* — by creating key pages, setting signature thresholds (e.g. "3 of 5"), and optionally delegating authority to other identities.

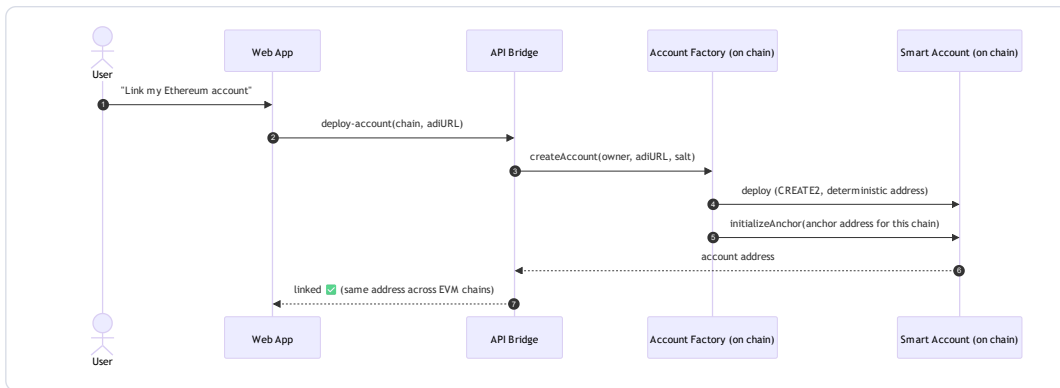


Walkthrough:

1. On the **World** page, the user edits the governance tree of an ADI.
2. They add **key pages** (each lists signer public keys) and set a **threshold** (how many signatures are required).
3. Optionally they **delegate**: allow another identity's key page to sign on their behalf — this creates a multi-hop "authority chain."
4. Each change is an Accumulate transaction (built by the bridge, signed by the Vault). From now on, every action obeys these rules — and the **governance proof (G1)** later proves the rules were followed.

3. Linking external-chain accounts

Plain English: For each blockchain the user wants to act on, Certen deploys a **smart account** controlled by the user's Accumulate identity. Thanks to clever address math (CREATE2), the account has the **same address on every EVM chain**.

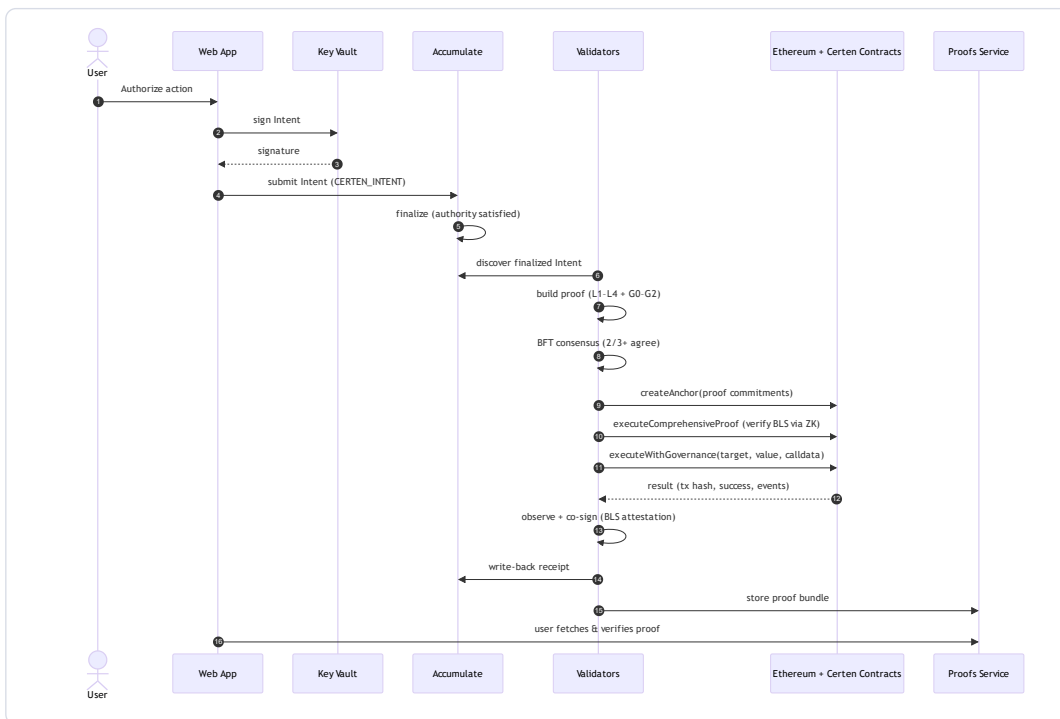


Walkthrough:

1. User picks a chain to link.
2. The **API Bridge** calls that chain's **Account Factory**.
3. The factory deploys a **smart account** at a **predictable address** (it can even be known *before* deployment) and binds it to that chain's **Anchor** contract.
4. The account is now ready to receive proof-authorized instructions. For non-EVM chains the same idea applies with each chain's native account model.

4. A single cross-chain action (the core flow)

Plain English: The headline capability. The user authorizes one action ("send 100 tokens on Ethereum"); Certen proves it was authorized and executes it, then files a receipt.



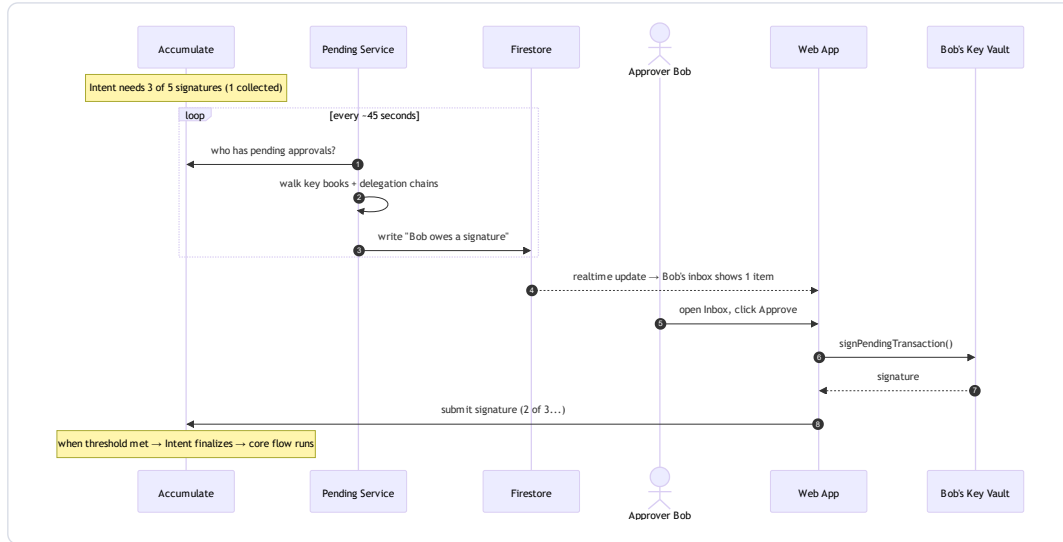
Walkthrough (the 3 on-chain contract calls in step "execute" matter):

1. User authorizes; the **Intent** is recorded and finalized on Accumulate.
2. Validators **discover** it, **build the proof**, and **agree** via consensus.
3. On the target chain, three contract calls happen in order:
 - o **createAnchor** — record the proof's fingerprints (commitments).
 - o **executeComprehensiveProof** — the **Anchor** asks the **BLS Verifier** to confirm (via a tiny ZK proof) that 2/3+ validators signed.
 - o **executeWithGovernance** — the **Account** checks the proof one more time (including the *execution tie*) and performs the action.
4. Validators **watch** the result, **co-sign** it, **write a receipt** back to Accumulate, and **archive** the full proof in the Proofs Service.

5. The user (or any auditor) can pull the proof and verify it independently.

5. A multi-signature approval

Plain English: When a key page requires several signatures (e.g. 3-of-5), the **Pending Service** notices who still needs to sign and notifies them. Each approver signs from their own Key Vault. Only when the threshold is met does the Intent finalize and the core flow (above) proceed.

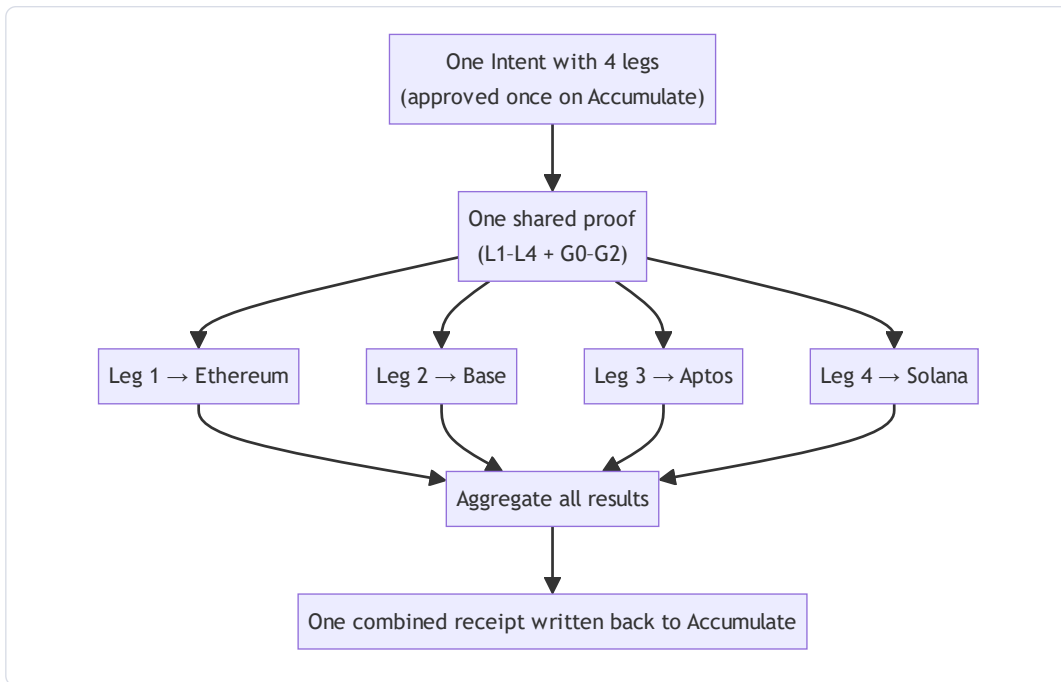


Walkthrough:

1. An Intent sits **pending** because it lacks enough signatures.
2. The **Pending Service** polls **Accumulate** every ~45s, walks each user's key books **and delegation chains** (up to 20 hops), and figures out exactly who can and still needs to sign.
3. It writes those "to-do" items to **Firestore**, which the web app subscribes to — so approvers see a **live inbox** without refreshing.
4. Each approver signs from their own Vault. When the threshold is reached, the Intent finalizes and the validator network takes over.

6. A multi-leg cross-chain action

Plain English: One approval, several destinations (e.g. "pay these 4 vendors on 4 different chains"). Certen proves the single intent once, executes each "leg" on its chain, and aggregates all the results into one combined receipt.

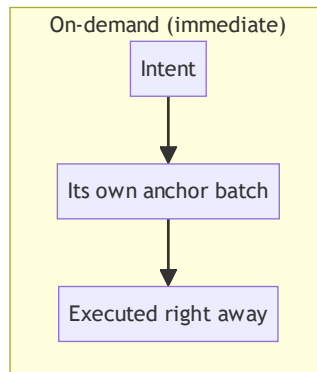
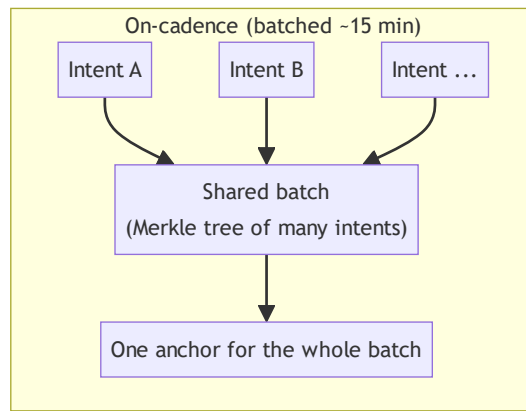


Walkthrough:

1. The user approves a **single** multi-leg intent.
2. The validators build **one** proof; each chain's **Anchor** receives the legs destined for it.
3. Each leg executes on its own chain and is observed independently.
4. All leg results are **aggregated** and written back to Accumulate as one receipt — so the audit trail shows the whole operation as a unit.

7. On-demand vs on-cadence proofs

Plain English: Two speed/cost tiers. **On-demand** = execute immediately (premium). **On-cadence** = batch many actions together every ~15 minutes and anchor them as a group (cheaper, because the on-chain cost is shared).

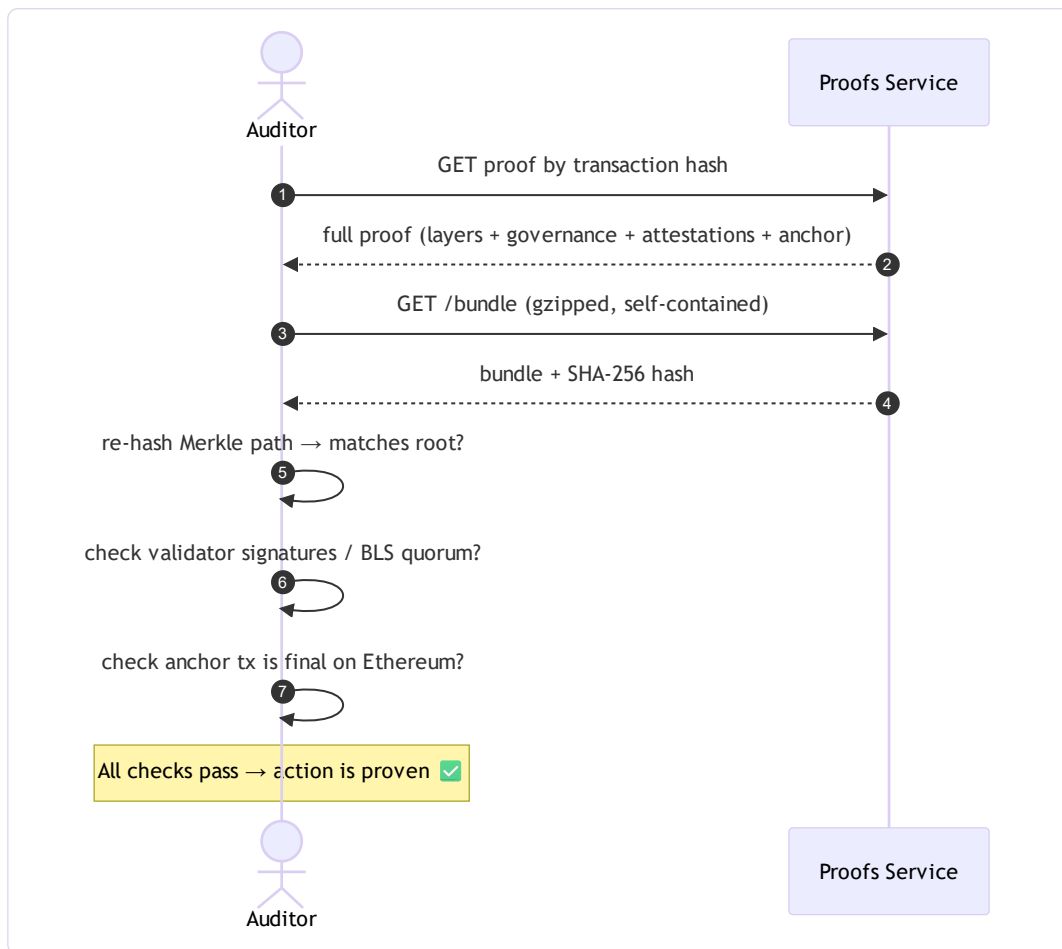


Walkthrough:

- The choice is set per intent (`proof_class`).
- **On-cadence** intents are collected into a **batch**, combined into a single **Merkle tree**, and anchored once — each intent gets a small **Merkle inclusion proof** showing it was in that batch.
- **On-demand** intents skip the wait and get their own immediate anchor.
- For on-demand intents, Certen requires a full **consensus-bound chained proof** (with bounded retries if a piece isn't ready yet) rather than silently downgrading to a weaker proof.

8. Viewing & independently verifying a proof

Plain English: Anyone with a proof can check it themselves — no need to trust Certen. The Proofs Service can hand out a self-contained "bundle" that an auditor can verify offline.



Walkthrough:

1. Look up a proof by its Accumulate transaction hash (or proof ID).
2. Download the **bundle** — it contains all four proof components plus the validator attestations.
3. Re-compute the Merkle path, verify the validator signatures (or the aggregated BLS quorum), and confirm the anchor transaction is finalized on the target chain. If everything matches, the action is mathematically proven.

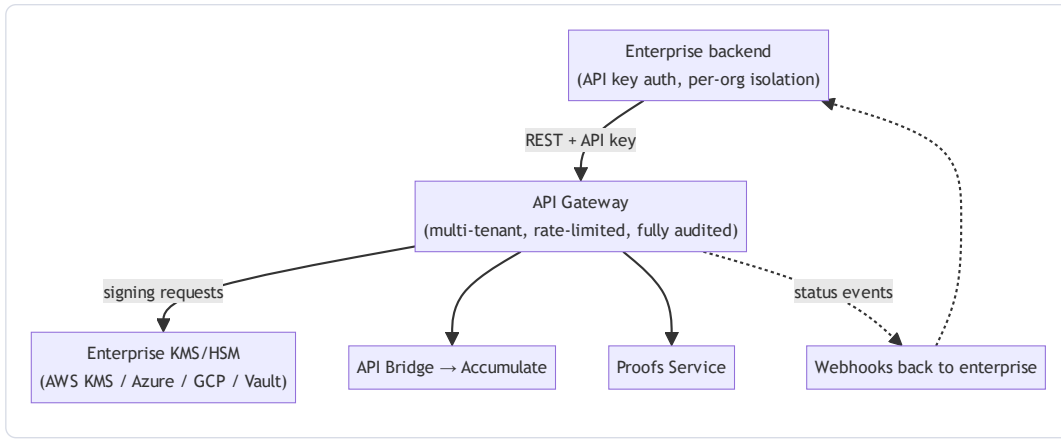
The web app's **Vault** page does this visually (Merkle tree explorer), but the same checks can be scripted by any third party.

9. Enterprise integration via the API Gateway (the full lifecycle)

Sections 1–8 told the story through Certen's **own web app and Key Vault** — the way a person clicks through it. An enterprise never touches that UI. It drives the **exact same system from its own backend**, over one authenticated REST API: the **API Gateway** (the "Stripe for Governance" surface). This section walks the **whole lifecycle again, end to end, but from the enterprise's side** — the same six stages, expressed as API calls and webhooks instead of clicks.

The two big differences from the web-app flow:

- **No browser wallet.** Signing is delegated to the enterprise's existing **key-management provider** (AWS KMS, Azure Key Vault, GCP KMS, HashiCorp Vault). Private keys never leave the enterprise's HSM/KMS.
- **No inbox to watch.** Instead of the Pending Service pushing to a UI, the Gateway fires **webhooks** at the enterprise's systems on every status change.

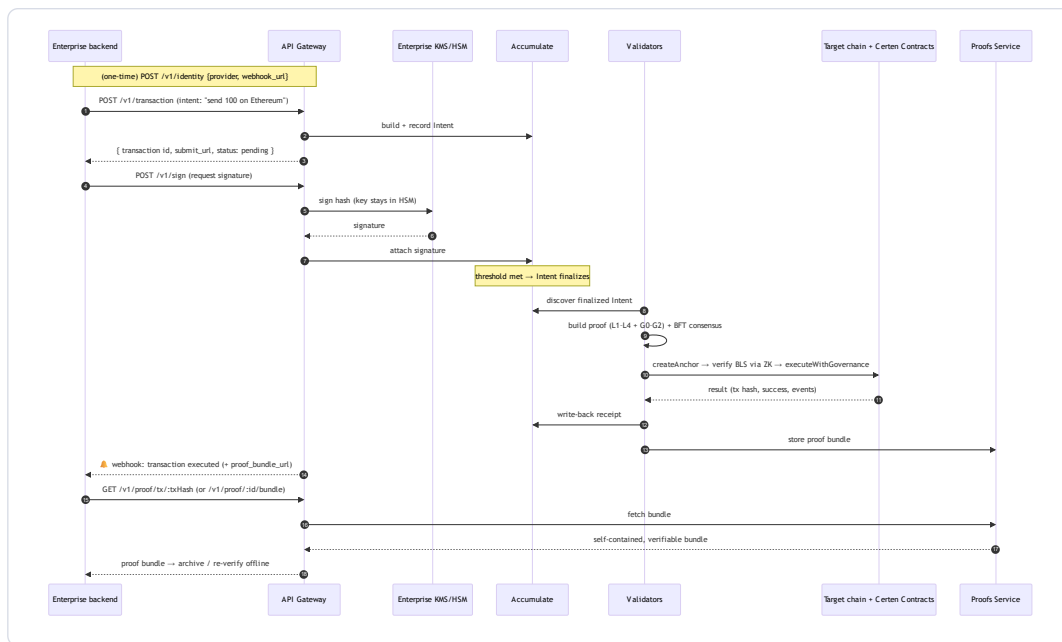


9a. The enterprise lifecycle, stage by stage

Stage (mirrors \$1–8)	API Gateway call(s)	What it does
Authenticate	every request carries an API key	Scopes the org, enforces rate limits, writes the audit log
Onboard an identity (\$1)	<code>POST /v1/identity</code> (with <code>provider</code> + <code>webhook_url</code>)	Orchestrates ADI + key book + credits and registers the signing provider in one call
Set up governance (\$2)	<code>POST /v1/governance</code> , <code>POST /v1/governance/:id/signature</code> , <code>GET /v1/governance/:id</code>	Create/update key pages, thresholds (m-of-n), delegation
Link chain accounts (\$3)	part of identity orchestration (<code>GET /v1/identity/:id?include=accounts</code>)	Deploys the smart accounts and reports their addresses
Submit an action (\$4)	<code>POST /v1/transaction</code> → returns a <code>submit_url</code>	Builds the Intent and submits it to Accumulate
Sign (\$4–5)	<code>POST /v1/sign</code> → <code>POST /v1/sign/:id/signature</code> , or <code>POST /v1/transaction/:id/signature</code>	Signing happens inside the enterprise's KMS , not a browser
Multi-sig approvals (\$5)	<code>GET /v1/pending</code> + webhooks	Each required approver's system is notified and signs in turn
Track to completion (\$4)	<code>GET /v1/transaction/:id</code> + webhooks	Status moves pending → executed; returns a <code>proof_bundle_url</code>
Fetch & verify proof (\$8)	<code>GET /v1/proof/:id</code> , <code>GET /v1/proof/:id/bundle</code> , <code>GET /v1/proof/tx/:txHash</code> , <code>GET /v1/proof/:id/custody</code>	Pull the verifiable bundle for compliance/audit
Portfolio / reporting	<code>GET /v1/portfolio</code> , <code>GET /v1/transactions</code>	Balances and history across all linked chains

9b. The core action, end to end, over the API

This is the enterprise mirror of the **core flow in \$4** — same proof, same validators, same on-chain execution, just driven by API calls and webhooks.



Walkthrough:

- One-time setup.** `POST /v1/identity` with a `provider` (the enterprise's KMS) and a `webhook_url` creates the ADI, governance, credits, and chain accounts, and wires up where status events should be delivered.
- Submit.** `POST /v1/transaction` builds and records the Intent; the response includes the transaction `id` and a `submit_url`, and an initial `pending` status.
- Sign in the HSM.** `POST /v1/sign` (then `/v1/sign/:id/signature`) has the Gateway ask the enterprise's **KMS** to sign — the private key never leaves the enterprise boundary. For additional approvers, each signs the same way.
- Threshold & hand-off.** When the key page's threshold is met, the Intent finalizes on Accumulate and the **validator network takes over** — identical to §4 from here on (discover → prove → consensus → anchor → execute → observe → write-back).
- Webhook, not polling.** The Gateway fires a **webhook** when the transaction reaches a terminal state, carrying a `proof_bundle_url`. (Systems that prefer polling can `GET /v1/transaction/:id` instead.)
- Fetch & verify.** The enterprise pulls the **proof bundle** and can verify it **offline**, exactly as an auditor does in §8 — re-hash the Merkle path, check the BLS quorum, confirm the anchor tx is final on chain. `GET /v1/proof/:id/custody` additionally returns the chain-of-custody record for compliance.

9c. Why this matters for enterprises

- **Same guarantees, zero UI.** The enterprise gets the identical cryptographic proof a web-app user gets — but fully programmatically, fit for CI/CD pipelines, treasury systems, or an internal approval portal.
- **Keys stay home.** Signing is delegated to the org's own KMS/HSM; Certen never holds enterprise keys.
- **Audit-ready by construction.** Every call is on the Gateway's audit log, and every executed action yields an independently verifiable proof bundle.

The **API Bridge** underneath is the lower-level, Accumulate-native service; the **Gateway** wraps it with auth, multi-tenancy, KMS signing, webhooks, and audit — everything an enterprise needs to treat Certen as infrastructure.

Continue to [04 — Proof Types & Their Parts](#) →

04 — Proof Types & What They're Made Of

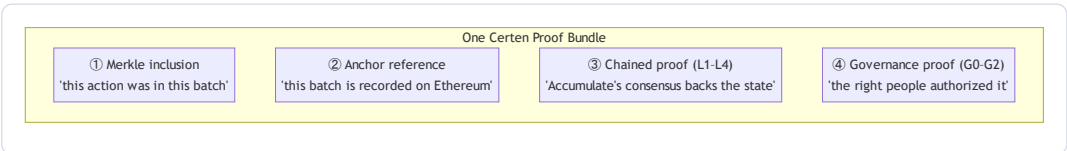
[← Back to index](#)

This document answers: **what is a Certen "proof," what kinds are there, and what parts does each one contain?**

1. What "proof" means here (and why it's special)

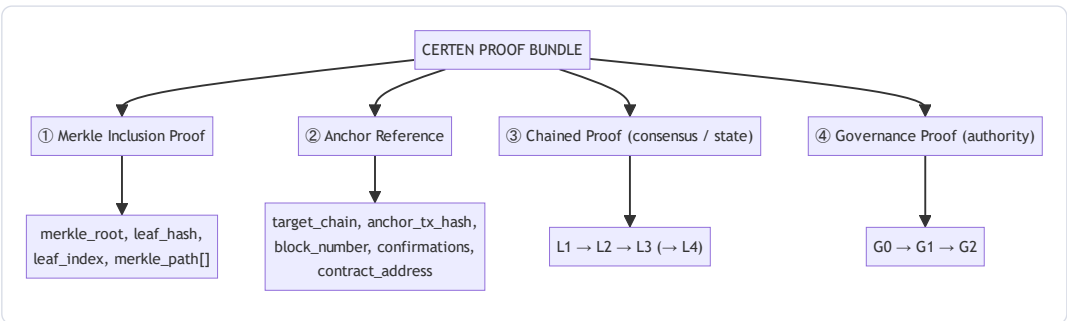
A Certen proof is **cryptographic evidence** — a package of numbers that lets *anyone* re-check that something is true, using only math. It does **not** ask you to trust Certen, a validator, or a server.

A good analogy: a **sealed certified-mail envelope** where every seal can be independently inspected:



Every complete proof has four components. Two of them (the chained proof and the governance proof) are themselves made of multiple *layers*. We'll unpack each.

2. The four components of a complete proof

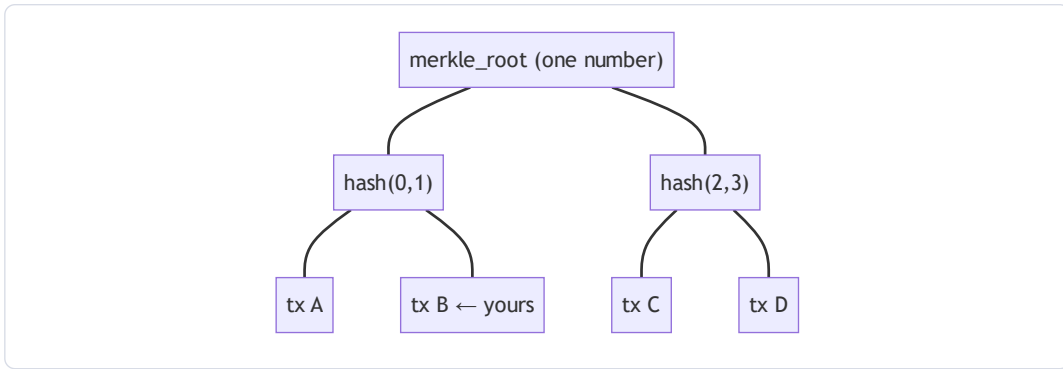


#	Component	Plain-English question it answers	Key fields
①	Merkle inclusion	"Was this exact transaction part of the anchored batch?"	<code>merkle_root</code> , <code>leaf_hash</code> , <code>leaf_index</code> , <code>merkle_path[]</code>
②	Anchor reference	"Where on an external chain is the fingerprint of that batch recorded, and is it final?"	<code>target_chain</code> , <code>anchor_tx_hash</code> , <code>anchor_block_number</code> , <code>confirmations</code> , <code>contract_address</code>
③	Chained proof (L1–L4)	"Does Accumulate's own consensus vouch for the state behind this?"	layered receipts (see §4)
④	Governance proof (G0–G2)	"Did the legitimately-authorized parties approve it?"	layered authority data (see §5)

3. Component ① & ②: Merkle inclusion + Anchor reference

Merkle inclusion — "it's in the batch"

A **Merkle tree** lets you prove one item belongs to a large set by revealing only a short path of hashes, not the whole set.



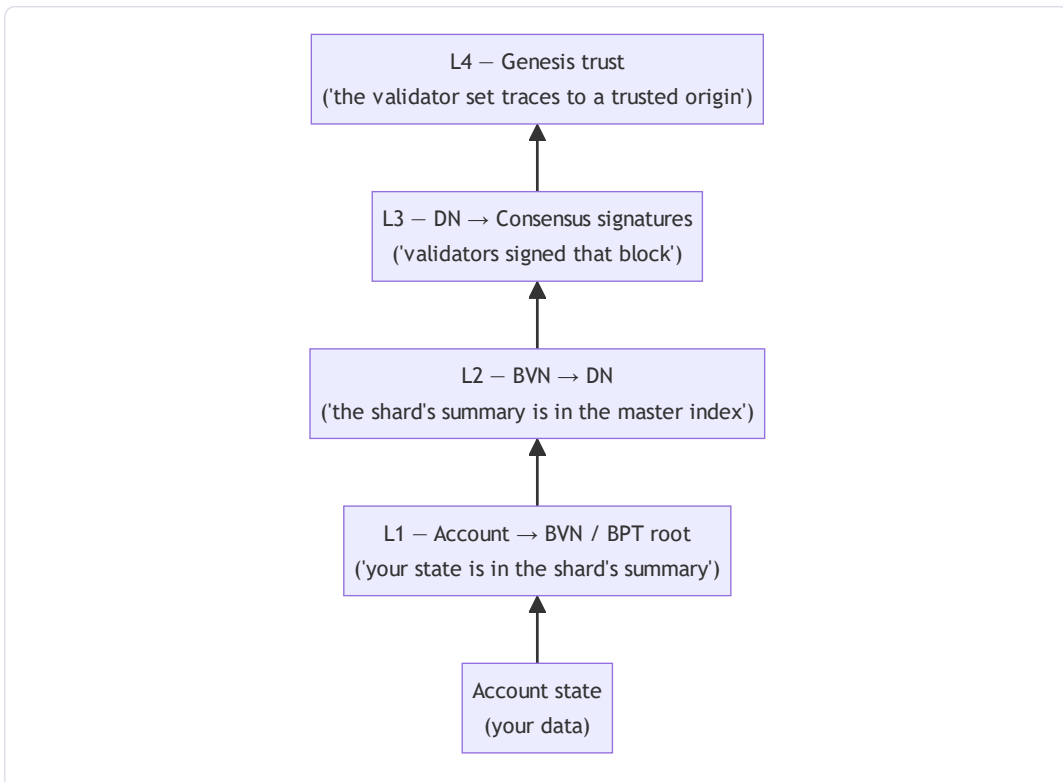
- Your transaction is a **leaf** (`leaf_hash`) at position `leaf_index` .
- The `merkle_path` is the handful of sibling hashes needed to climb to the `merkle_root` . Re-hash along the path; if you reach the same root, your transaction was provably included — and nothing was altered.

Anchor reference — "the batch is on a real chain"

This component points to the **anchor transaction** on the external chain that recorded the batch's `merkle_root` (plus other commitments). It includes which chain, the transaction hash, block number, how many confirmations it has, and whether it's final. That's what makes the proof **publicly checkable** by anyone with a block explorer.

4. Component ③: the Chained Proof (L1–L4) — consensus & state

This is the ladder that ties an action all the way back to Accumulate's consensus. Think of it as proving a fact "all the way up the chain of command."



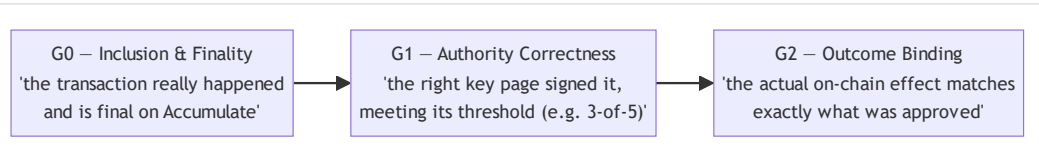
Layer	Proves	Contains (key parts)
L1 Account → BVN	Your account's state is part of the shard's official state summary (the app_hash / BPT root)	bvn_partition , source_hash , target_hash , receipt entries
L2 BVN → DN	That shard summary is anchored into Accumulate's master index (Directory Network)	bvn_root , dn_root , anchor_sequence
L3 DN → Consensus	The master block was signed by the validators (the consensus that makes it official)	dn_block_hash , dn_block_height , consensus timestamp, signatures
L4 Genesis trust	The signing validator set itself traces back to a trusted starting point	validator-set / genesis linkage

Honest status note: L1–L3 are live and enforced, and the **L4 consensus binding is live and fail-closed today** (a proof that doesn't tie back to Accumulate's consensus is rejected outright). The remaining piece of L4 — the *full* genesis-to-now validator-set proof, with per-signature and voting-power checks — depends on capabilities the **current Accumulate mainnet does not yet expose** to lite clients. Certen is **working directly with the Accumulate core developers** to add that ability at the protocol level; once it lands, the complete L4 validator-set proof can be enforced end-to-end. The accurate one-liner: *"L1–L3 done; L4 binding live and fail-closed; the full L4 validator-set proof is in progress with Accumulate core."*

Why "chained"? Each layer is cryptographically linked to the next. If any single link is faked, the chain breaks and verification fails — you can't forge an intermediate step without breaking the whole thing.

5. Component ④: the Governance Proof (G0–G2) — authority

This proves the action was **legitimately authorized** under the identity's own rules. Three levels, increasing in strength:

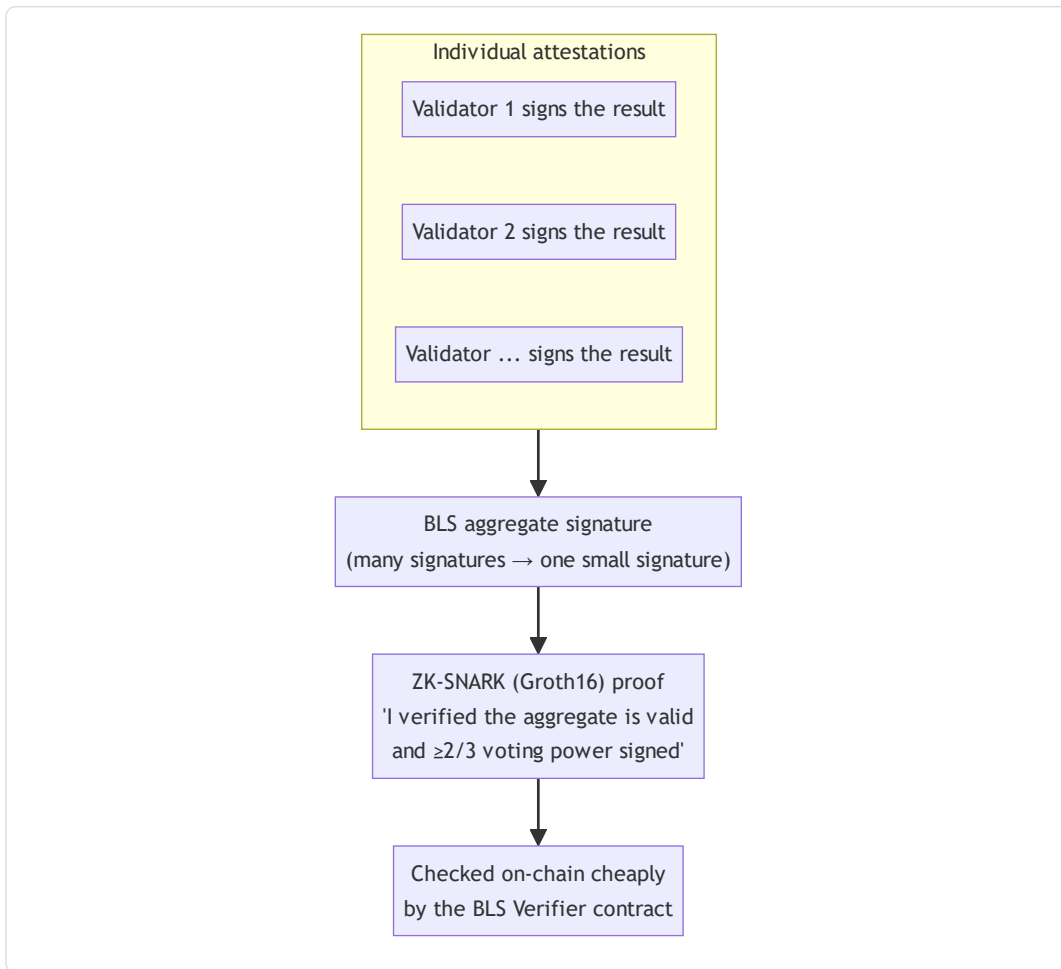


Level	Plain meaning	Contains (key parts)
G0 Inclusion & finality	The transaction is in a block and that block is final/anchored	block height, finality timestamp, anchor height, is_anchored
G1 Authority correctness	The approving key page existed, the signers were on it, and the threshold (m-of-n) was met	authority URL, key-page state (version, keys, threshold m/n), signature count
G2 Outcome binding	The executed transaction's payload and effects match what was authorized (so nothing was swapped)	payload verification (computed vs expected hash), effect verification (success/revert/events)

Why three levels? Different use cases need different assurance. A read-only or low-value action might be fine at **G1**; a value-moving transfer requires **G2** (outcome binding) so the on-chain effect is provably the one that was approved.

6. The signature & attestation pieces (how the group vouches)

Two cryptographic mechanisms make the validators' collective endorsement both **trustworthy** and **cheap to check on-chain**:

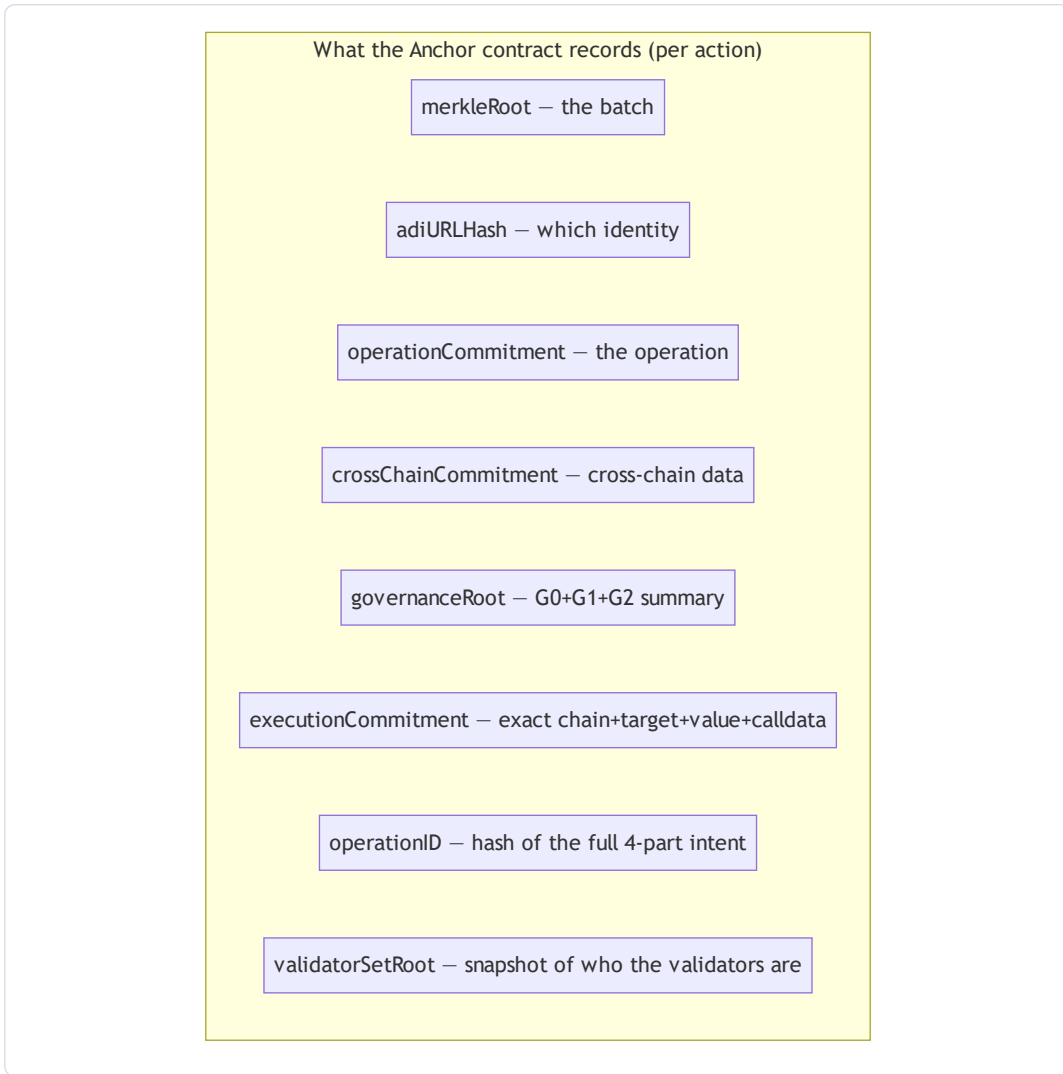


Piece	What it is	Key parts
Validator attestation	One validator's signature over the result hash	<code>validator_id</code> , public key, signature, <code>signature_valid</code>
BLS aggregate	All those signatures combined into one (using BLS12-381)	aggregate signature, aggregate public key, validator bitfield, voting-power %
ZK-SNARK (Groth16)	A tiny proof that "the aggregate is valid and met the 2/3 threshold," so the chain doesn't redo expensive pairing math	Groth16 proof (a, b, c), public inputs: message hash, voting power signed/total, threshold

Why this matters: checking dozens of individual signatures on Ethereum would be expensive. BLS shrinks them to one; the ZK proof shrinks the *verification* down to something a contract can confirm cheaply. That's how Certen keeps on-chain costs low while preserving "2/3 of validators vouched for this."

7. The fingerprints ("commitments") an Anchor stores

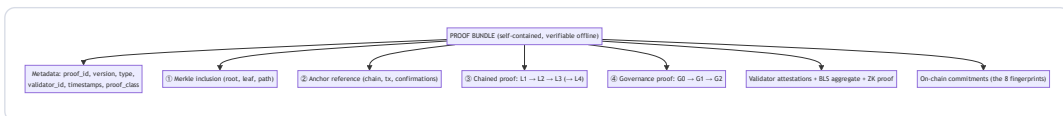
When the action is anchored on-chain, the contract doesn't store the whole proof (too big/expensive). It stores compact **commitments** — fixed-size fingerprints — and the BLS signature is bound to all of them at once:



These are the on-chain anchors of the off-chain proof. The **executionCommitment** and **operationID** are what make a proof impossible to replay for a different action; the **validatorSetRoot** is what makes it impossible to replay with a stale set of validators. (Details in [06 — Contracts](#).)

8. The whole proof, assembled

Putting all the parts together, here's what a complete proof bundle looks like:



The **Proofs Service** stores all of this in a database and can serve it as a gzipped bundle with a SHA-256 hash, so an auditor can verify integrity and then re-check each component independently.

9. Quick reference: every proof artifact type

For the more technical readers, these are the distinct artifact types the system produces and stores:

Artifact	What it captures
Merkle inclusion proof	Membership of a tx in a batch
Anchor record / reference	The external-chain transaction recording a batch
Chained proof (L1–L4)	Accumulate consensus/state ladder
Governance proof (G0–G2)	Authorization correctness
Validator attestation (Ed25519)	One validator's endorsement of a proof
External chain result (the "execution proof")	The observed on-chain outcome (tx, block, status, events, state root)
BLS attestation	One validator's BLS signature on the result
Aggregated BLS attestation	Combined quorum signature + threshold met
Proof bundle	Everything above, packaged for offline verification

Continue to [05 — How Proofs Are Used](#) →

05 — How Proofs Are Used: Where & When

[← Back to index](#)

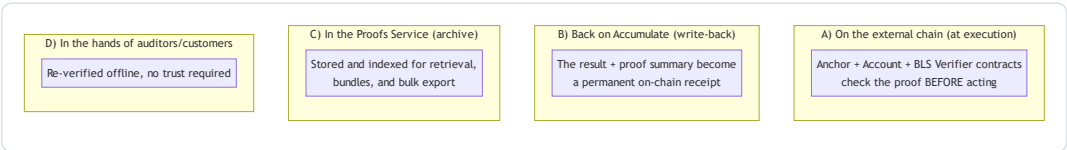
The previous doc explained *what proofs contain*. This one explains *how they're used in practice* — the full life of a proof, and the exact moments where each proof does its job.

1. The life of a proof (cradle to grave)



Each stage is a place where a proof (or part of one) is **used**. Let's go through the three that matter most: **execution-time on-chain use**, **write-back use**, and **after-the-fact verification use**.

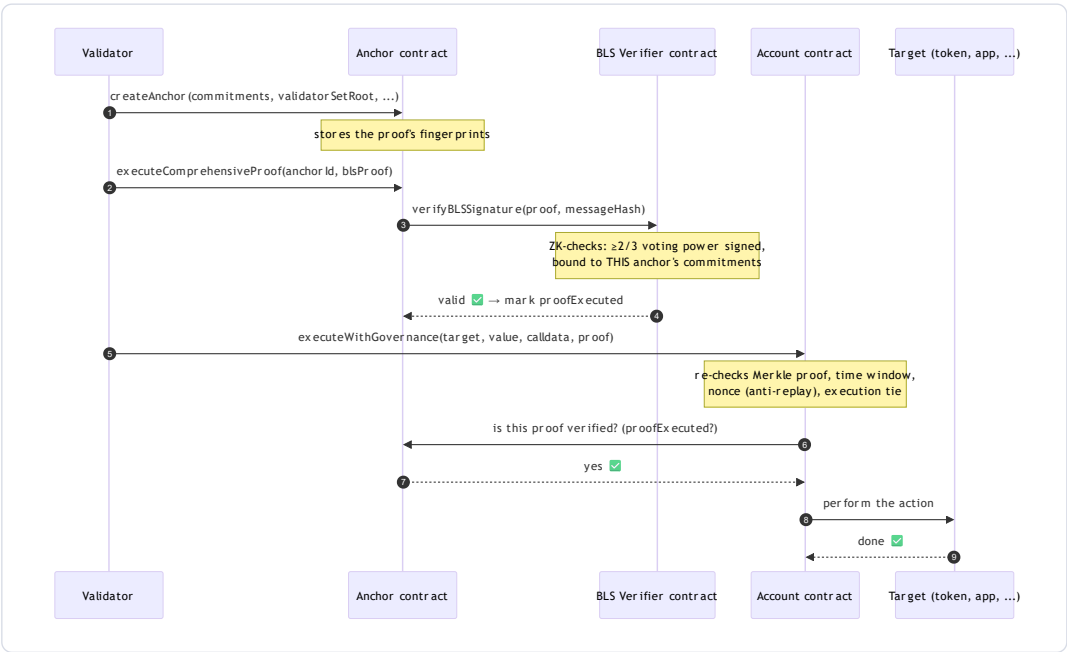
2. WHERE proofs are used — the four locations



Location	When	What the proof does there
A. External chain contracts	At the moment of execution	The proof is the key that unlocks action. No valid proof → nothing happens.
B. Accumulate (write-back)	Right after execution	The proof summary + result is recorded as the official receipt , closing the loop.
C. Proofs Service	Continuously	The proof is archived, indexed, and served so anyone can fetch it later.
D. Auditors / customers	Anytime afterward	The proof is independently re-verified with math alone.

3. Use A (the critical one): proofs gate on-chain execution

This is the heart of Certen's security. On the external chain, **three contracts check the proof in sequence**, and execution only proceeds if all checks pass.



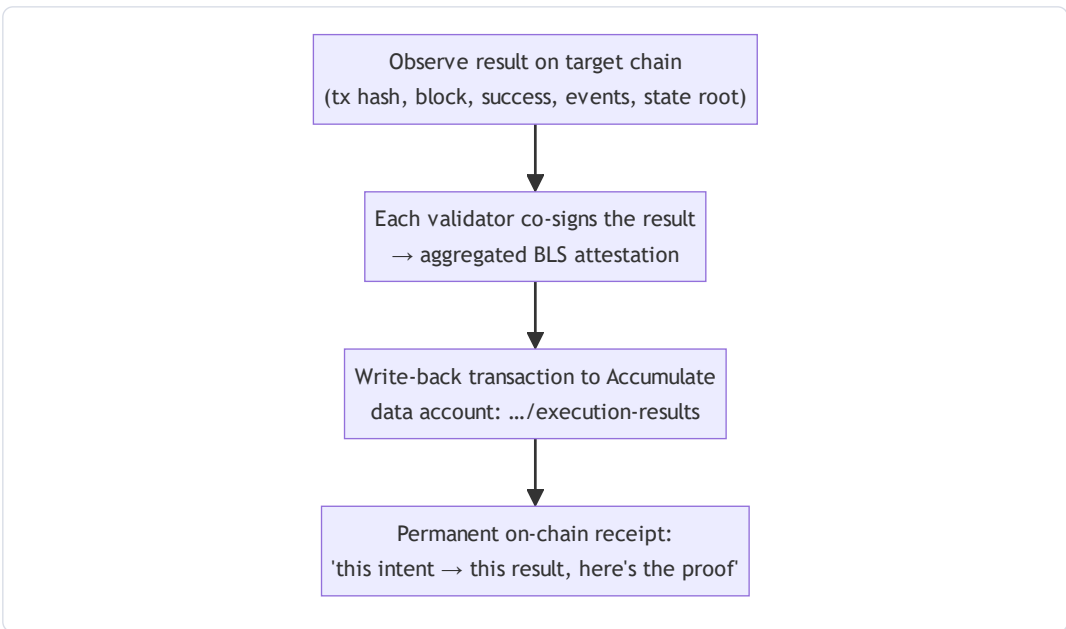
When each check happens and why:

Step	Proof part used	Guards against
<code>createAnchor</code>	The 8 commitments (Merkle root, execution commitment, operationID, validatorSetRoot, ...)	Recording an action that wasn't agreed; pins it to the exact operation
<code>executeComprehensiveProof</code>	BLS aggregate + ZK proof (component from 04 §6)	A minority of validators acting alone; only a 2/3 quorum passes
<code>executeWithGovernance</code>	Merkle inclusion + governance + execution tie	Replays, parameter-swapping, wrong-chain reuse, expired proofs

The **execution tie** (`executionCommitment`) check is worth repeating: the contract recomputes `hash(chainId, target, value, calldata)` and demands it match what the proof committed to. A proof authorizing "send 100 to Alice on Ethereum" **cannot** be reused to "send 1,000,000 to Mallory" or run on a different chain.

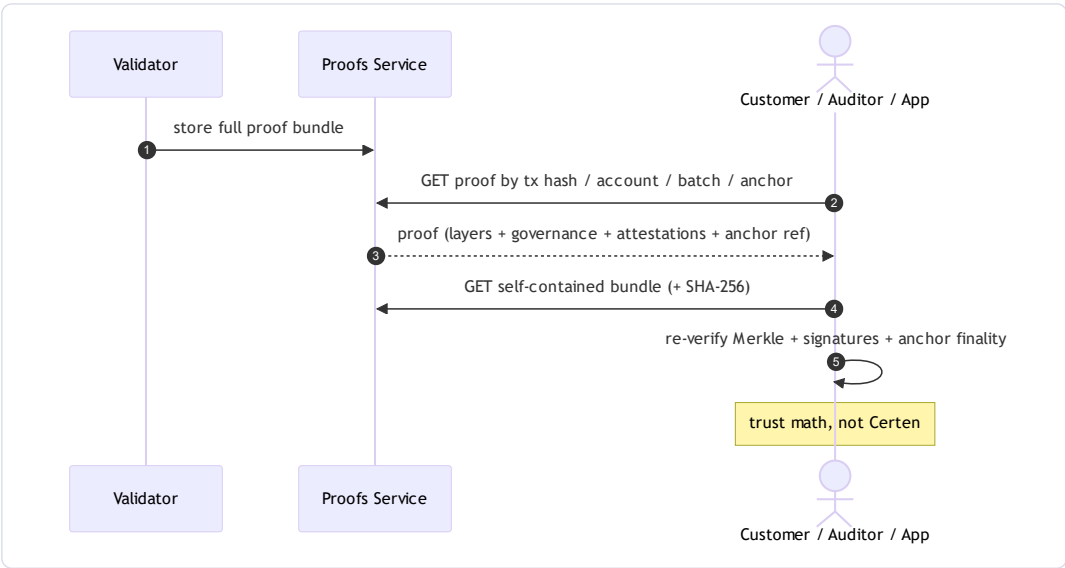
4. Use B: write-back proofs create the permanent receipt

After execution, the validators don't just walk away — they **observe** the result, **co-sign** it, and **write it back** to Accumulate. This write-back is itself proof-backed.



When: immediately after the target chain confirms the action. **Where:** the Certen network's own Accumulate data account (`acc://certen-protocol.acme/execution-results`). **Why it matters:** the original decision and its real-world outcome now live **together on the same trusted ledger** — a complete, tamper-evident audit trail from "we decided X" to "X happened, proven."

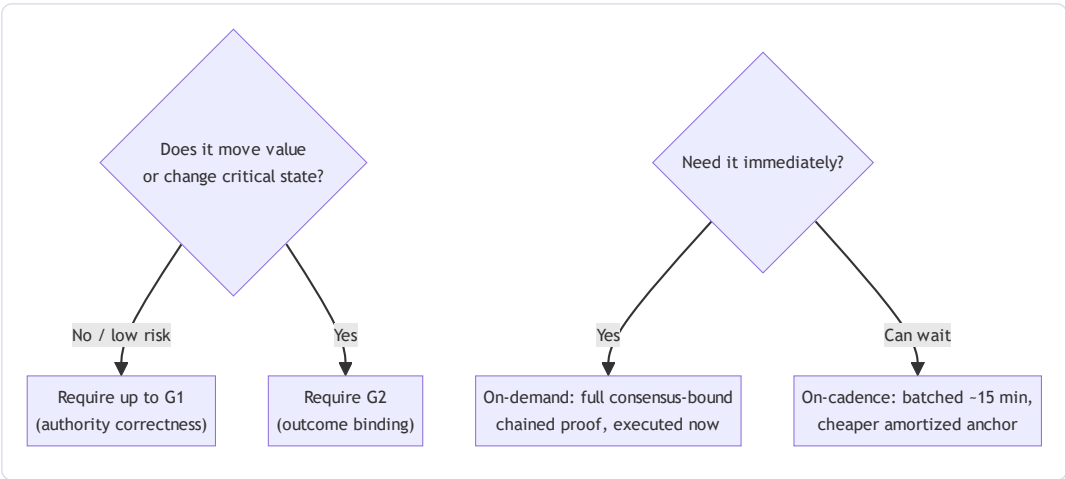
5. Use C & D: archive and independent verification



- **Archive (C):** the **Proofs Service** indexes every proof so it's findable by transaction hash, account, batch, or anchor — and offers bulk export for compliance teams.
- **Independent verification (D):** the bundle is **self-contained**. An auditor re-hashes the Merkle path, checks the validator/BLS signatures, and confirms the anchor transaction is final on the public chain. No part of this requires trusting Certen's servers.

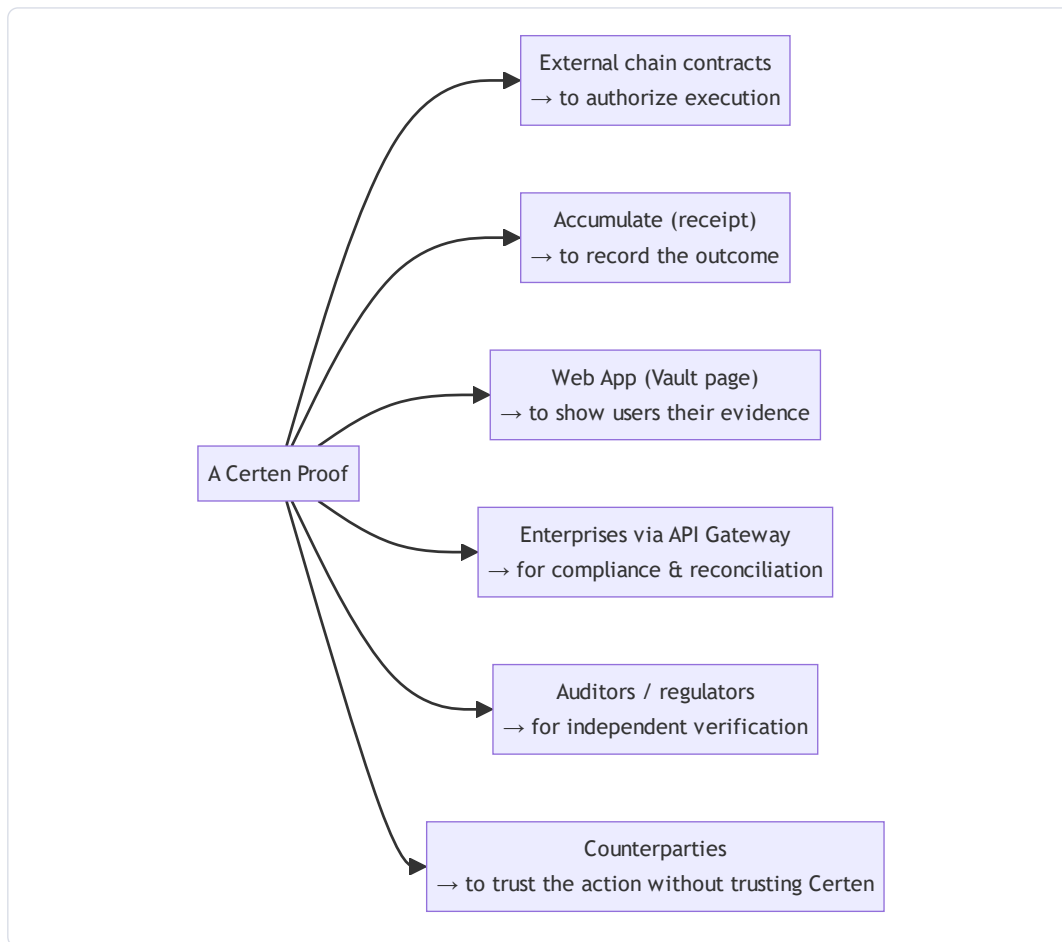
6. WHEN each proof level is required (the rules of thumb)

Not every action needs the maximum proof. The system matches proof strength to risk:



Situation	Proof requirement
Low-value / read-ish operation	Governance up to G1 may suffice
Value-moving transfer	G2 outcome binding required (effect must match approval)
Time-sensitive	On-demand with a full consensus-bound chained proof; bounded retries if a layer isn't ready, rather than downgrading
Cost-sensitive, batchable	On-cadence: combined into a Merkle batch, one shared anchor

7. Who consumes proofs, and why



The single most important point for founders: **the same proof serves all of these consumers**, because it's verifiable by anyone. That's the product's core value — *one piece of evidence that everyone, everywhere, can independently trust*.

Continue to [06 — The Smart Contracts Explained](#) →

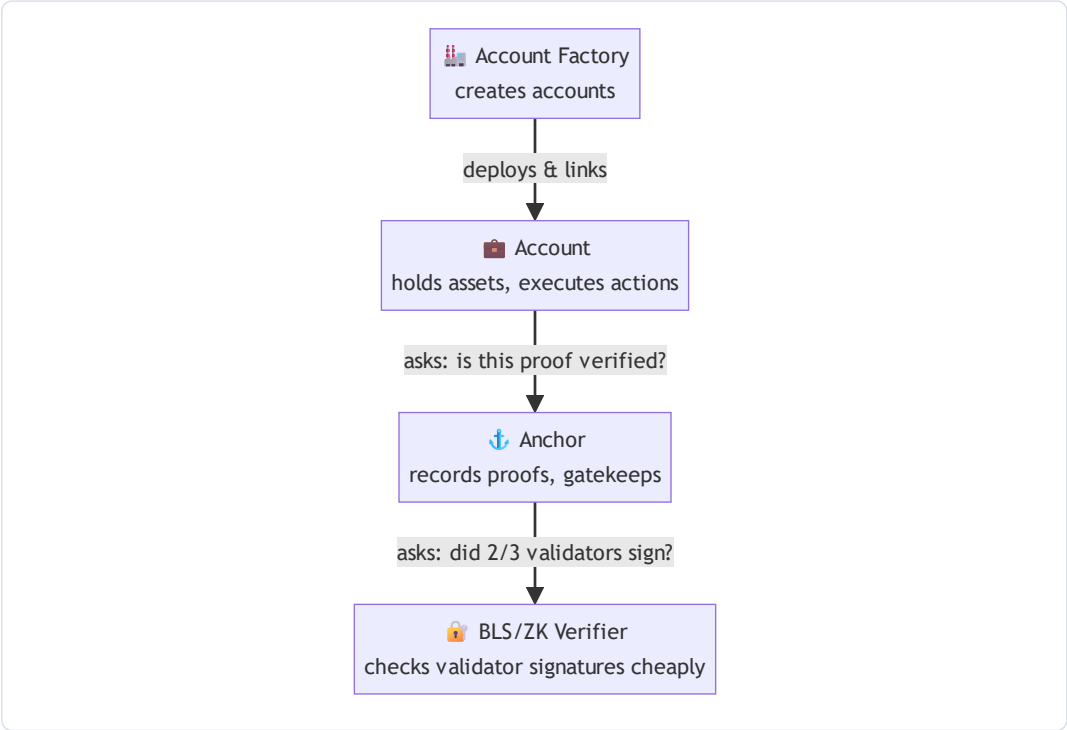
06 — The Smart Contracts Explained

[← Back to index](#)

This document explains the four on-chain contracts that make Certen work on each external chain: the **Account**, the **Account Factory**, the **BLS/ZK Verifier**, and the **Anchor**. Each section starts plain and adds detail.

The examples below use the **EVM (Solidity)** contracts, which run on Ethereum, Arbitrum, Base, Polygon, Hedera, etc. Non-EVM chains (Solana, Aptos, Sui, NEAR, TON, Cardano) have **functionally equivalent** contracts written in each chain's native language. The roles and checks are the same; only the code differs.

1. How the four contracts fit together



Contract	One-line job	Analogy
Account	The user's on-chain wallet that only acts on proof-authorized instructions	A safe that opens only with a verified order
Account Factory	Mass-produces accounts at predictable addresses	The locksmith that builds identical safes
Anchor	Records each proof's fingerprints and acts as the gatekeeper	The notary's logbook + the gate
BLS/ZK Verifier	Confirms a 2/3 validator quorum signed, cheaply	The signature-checking machine

2. The Account contract

What it does (plain)

It's a smart **wallet that belongs to an Accumulate identity (ADI)**. It can hold tokens and perform actions, but — crucially — it will only do so when shown a **valid governance proof**. The owner can't just move funds on a whim if the governance rules say otherwise.

It's built on the **ERC-4337 "account abstraction"** standard, which (among other things) allows gasless and flexible transaction flows.

Key functions

`executeWithGovernanceProof(target, value, data, proof)`
main path: act only with a valid ADI proof

`batchExecuteWithGovernanceProof(...)`
many actions, one proof

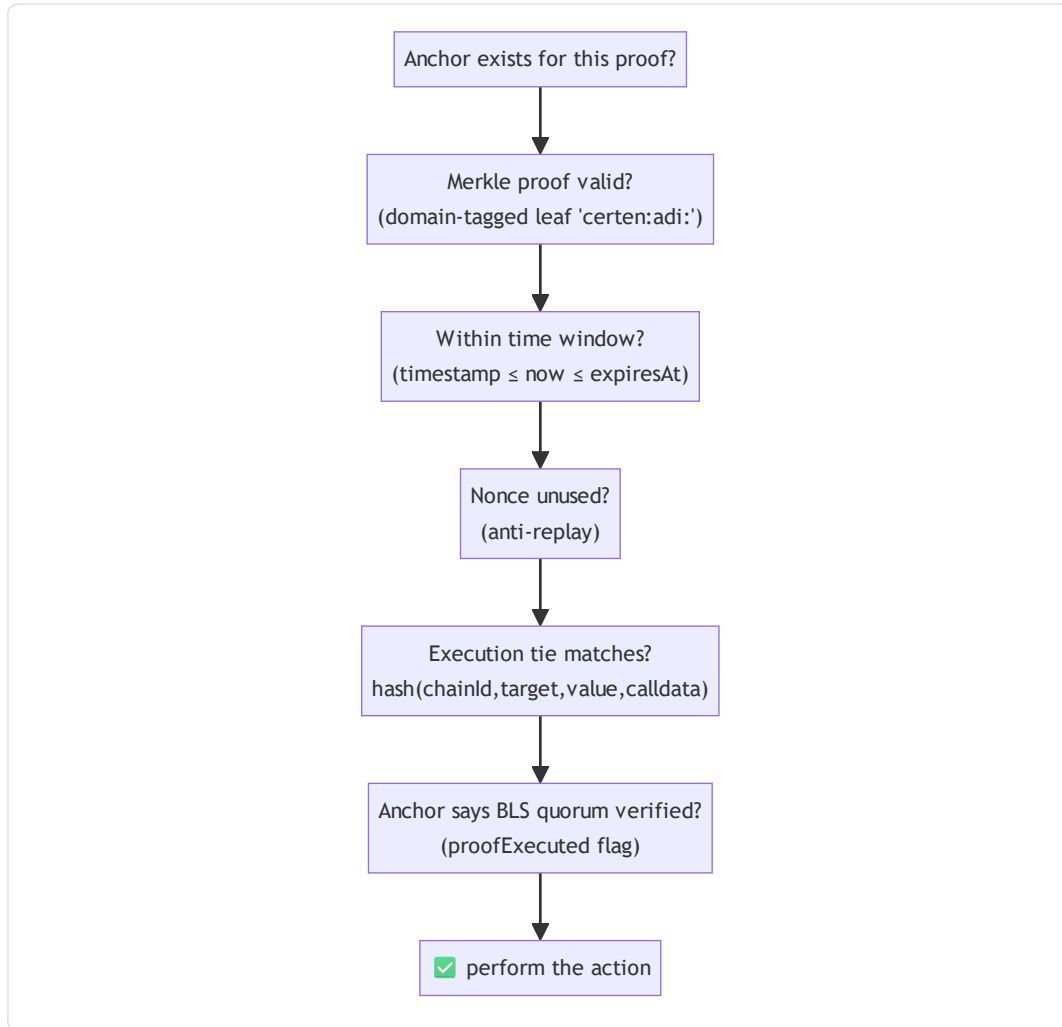
`executeGovernanceProofDirect(...)`
validator-initiated, no relay needed

`initializeAnchor(anchorAddress)`
one-time; bind to this chain's Anchor

`execute(target, value, data)`
owner-only escape hatch (testing/emergency)

What it checks before acting

When asked to execute with a proof, the Account verifies (in order):



- **Domain-tagged Merkle leaf:** the account rebuilds the leaf exactly as the anchor stored it (`keccak256("certen:adi:" + adiURLHash)`), preventing "leaf-swap" tricks.
- **Execution tie:** recomputes the fingerprint of the *actual* call and demands it equal what the proof committed to — the anti-parameter-swap guarantee.
- **Authority levels:** the account understands roles (`OPERATOR` , `MANAGER` , `ADMIN` , `ROOT`) and can require a minimum level per operation.

State it keeps

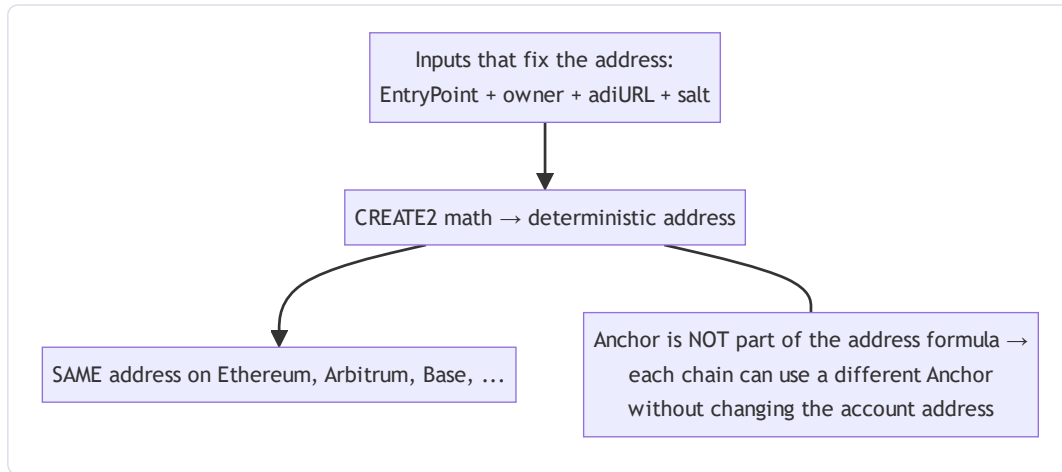
The ADI URL it belongs to, the linked Anchor contract, used-proof records (replay protection), per-operation nonces, and role information.

3. The Account Factory

What it does (plain)

It **deploys Account contracts** — and does so at a **predictable address**. For EVM chains, the same identity gets the **same account address on every chain**, which is a big usability win (one address to remember, derivable in advance).

The clever bit: address stability



- It uses **CREATE2** (a deterministic deployment method) via a standard keyless deployer, so the address can be computed **before** the account exists.
- The **Anchor address is deliberately excluded** from the address formula. That's why the same account address works across chains even though each chain has its own Anchor — the factory links the anchor *after* deployment via `initializeAnchor`, atomically in the same transaction (no front-running window).

Key functions

- `createAccount(owner, adiURL, salt)` — deploy one account.
- `createAccountIfNotExists(...)` — idempotent (returns existing if already there).
- `batchCreateAccounts(...)` — deploy many at once.
- `getAddress(...)` — predict the address without deploying.
- `setAnchorContract(...)` — set the chain-specific Anchor (owner only).

It also keeps a registry (ADI → account) so each identity gets exactly one account per chain.

4. The BLS/ZK Verifier

What it does (plain)

It answers one question cheaply: **"Did a 2/3 supermajority of Certen validators sign this?"** Checking many signatures directly on-chain would be slow and expensive, so instead the validators produce a **zero-knowledge proof (Groth16)** that the aggregate signature is valid and met the threshold, and this contract verifies that little proof.



What it checks

1. The proof's message hash equals the one the Anchor supplied (binding).
2. `signedVotingPower ≥ 2/3 × totalVotingPower` — using the **stored** threshold, not a caller-supplied one (so a caller can't lower the bar).
3. The Groth16 proof itself verifies against a **baked-in verification key** (immutable, audited, generated to match the prover exactly).

Why two addresses?

In the live config you'll see a **V2 adapter** and a **V2 generated** address. The "generated" contract is the raw ZK verifier (auto-generated to match the proving circuit bit-for-bit); the "adapter" wraps it with Certen's interface and the threshold/message-binding logic. The Anchor talks to the adapter.

5. The Anchor — the gatekeeper

What it does (plain)

The Anchor is the **central gatekeeper** on each chain. It:

1. **Records** each proof's fingerprints (commitments) when an action is anchored.
2. **Verifies** the validator quorum (by calling the BLS Verifier).
3. **Gatekeeps** execution — the Account asks the Anchor "is this proof really verified?" before acting.
4. **Maintains the validator registry** and a snapshot of the current validator set.

The three-call execution dance

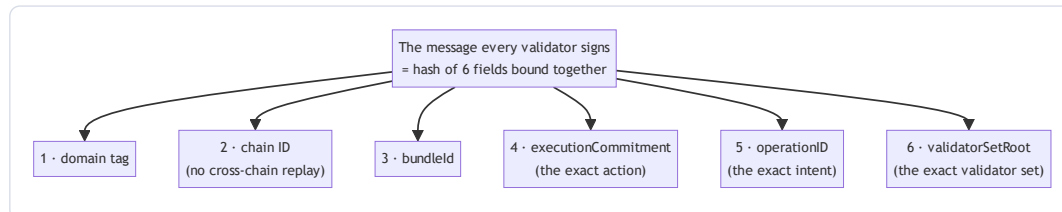
```
#mermaid-1782550560528{font-family:"trebuchet ms",verdana,arial,sans-serif;font-size:16px;fill:#333;}#mermaid-1782550560528 .error-icon{fill:#552222;}#mermaid-1782550560528 .error-text{fill:#552222;stroke:#552222;}#mermaid-1782550560528 .edge-thickness-normal{stroke-width:2px;}#mermaid-1782550560528 .edge-thickness-thick{stroke-width:3.5px;}#mermaid-1782550560528 .edge-pattern-solid{stroke-dasharray:0;}#mermaid-1782550560528 .edge-pattern-dashed{stroke-dasharray:3;}#mermaid-1782550560528 .edge-pattern-dotted{stroke-dasharray:2;}#mermaid-1782550560528 .marker{fill:#333333;stroke:#333333;}#mermaid-1782550560528 .marker.cross{stroke:#333333;}#mermaid-1782550560528 svg{font-family:"trebuchet ms",verdana,arial,sans-serif;font-size:16px;}#mermaid-1782550560528 :root{--mermaid-font-family:"trebuchet ms",verdana,arial,sans-serif;}Syntax error in textmermaid version 10.9.3
```

What it stores per action (the commitments from 04 §7)

`merkleRoot`, `adiURLHash`, `operationCommitment`, `crossChainCommitment`, `governanceRoot`, `executionCommitment`, `operationID`, the Accumulate block height, plus flags (`valid`, `proofExecuted`, `governanceExecuted`) and the governance level achieved.

How the Anchor makes a signature impossible to replay

The Anchor binds **every validator signature to six independent fields at once** — so a signature is only valid for one exact action, on one exact chain, by one exact set of validators:



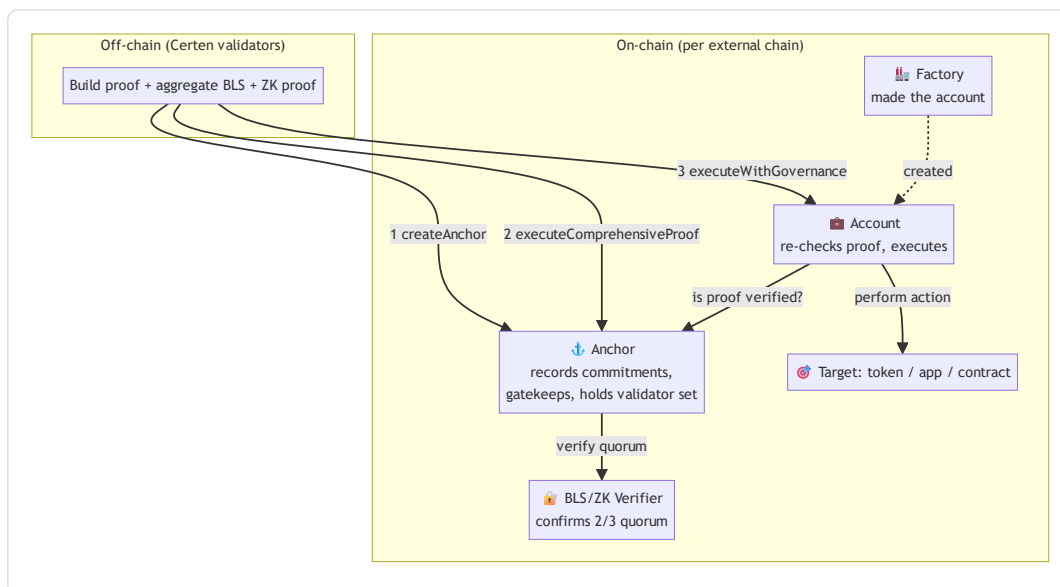
Each field closes off a different attack:

- The **chain ID** stops a signature from being replayed on a different chain.
- The **executionCommitment** stops the action's parameters (target/value/calldata) from being swapped.
- The **operationID** ties the signature to one specific approved intent.
- The **validatorSetRoot** ties it to the exact validator set: if validators are added/removed or the threshold changes, the snapshot changes and **old signatures instantly stop verifying** — no replay from a stale validator set.

Key functions

- `createAnchor(...)` / `createAnchorWithLegs(...)` — record an action (single or multi-leg). Validator-only.
- `executeComprehensiveProof(anchorId, blsProof)` — verify the quorum via the BLS Verifier; flips `proofExecuted`.
- `verifyProof(anchorId, merkleProof, leaf)` — Merkle check used by Accounts.
- `registerValidator` / `removeValidator` / `setThreshold` — manage the validator set (each recomputes the validator-set snapshot).
- `getValidatorSetRoot()` / `getExecutionCommitment(anchorId)` — read helpers.

6. The complete on-chain picture



The one-sentence summary of all four contracts: the **Factory** builds an **Account** that refuses to act until the **Anchor** — backed by the **BLS/ZK Verifier** — confirms that a 2/3 validator quorum cryptographically approved *this exact action*, traceable all the way back to a finalized, properly authorized decision on Accumulate.

7. Caveats & current status (be honest with the team)

- **Non-EVM parity:** EVM chains run the full V6.1 binding. Non-EVM chains (Solana, Aptos, Sui, NEAR, TON, Cardano) have been brought to the same ("A+++") parity, each verified on-chain; a couple have known chain-specific limitations in *full* end-to-end runs that are tracked separately.
- **Proof depth:** on-chain checks rely on the off-chain proof. The deepest consensus layer (**L4** genesis/validator-set proof) has its consensus *binding* live and fail-closed today; the full validator-set proof is in progress with the Accumulate core developers (see [04 §4](#)).

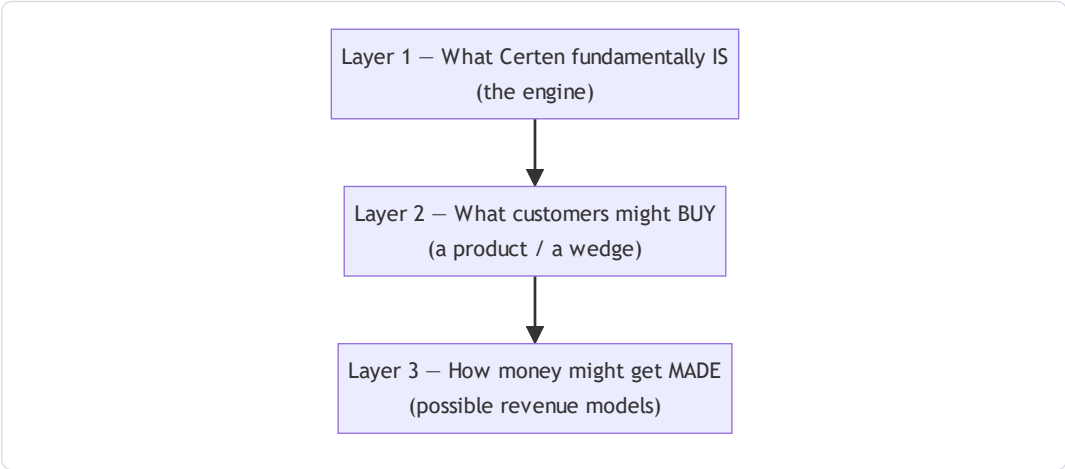
Continue to [07 — Possible Framing, and Monetization Paths](#) →

07 — Possible Framing, and Monetization Paths

[← Back to index](#)

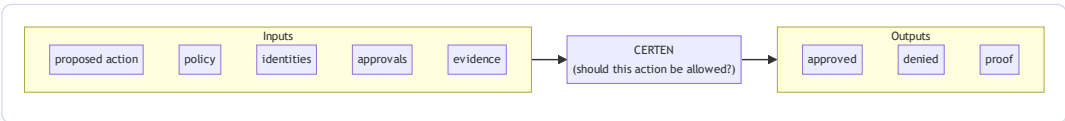
The earlier documents explain **how Certen works**. Think of this one as: *"here's what I think I've actually built, and a few directions it might point" — not a business plan.*

One framing I've found genuinely clarifying is to **keep three layers separate**, because they're easy to blur:



1. Useful framing for Certen

From the inside, the purest description I'd give is: **a trust-decision engine**.



At its core it answers one question — **"Should this action be allowed?"** — and emits a verifiable proof of the answer.

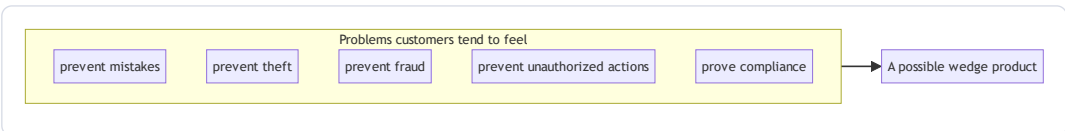
A thing worth noticing: from an engineering standpoint, **"governance" looks like just one application** of the engine rather than the engine itself. The same machinery could, in principle, serve treasury control, cross-chain operations, lending controls, AI-agent guardrails, and more — they differ mainly in which policy and actions get plugged in.

If I had to compress what the A+++ 6.1 work is really demonstrating into one line, it would be:

Execution is conditional on proof.

2. Layer 2 — what customers might actually buy

The engine is hard to sell as an engine. People don't tend to wake up wanting "trust infrastructure" or "a policy engine" — they tend to want a problem to go away:



A **narrow, instantly legible wedge** might land better than the full vision. One candidate that's easy to explain is **Governance & Change Control**:

- software release / protocol-upgrade approvals
- treasury withdrawals
- key rotations
- validator additions

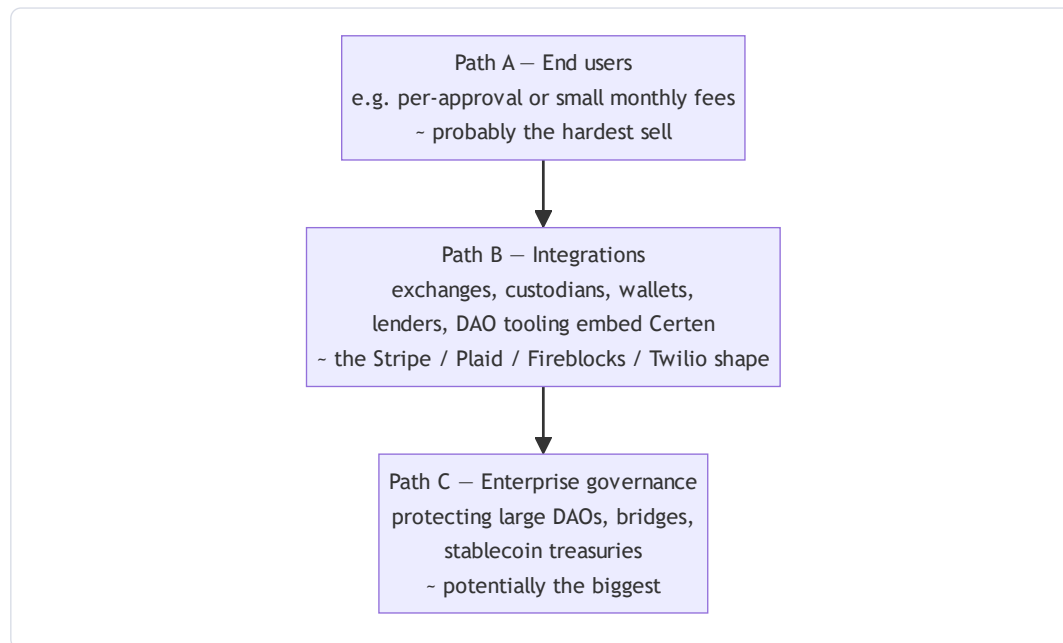
A phrasing I've toyed with for that wedge:

Cryptographic Change Control for Digital Assets and Autonomous Systems.

The interesting part, if a wedge like that works, is that **customers may then discover** they've bought something more general — a trust-decision engine — which is where a larger platform story could open up.

3. Layer 3 — where the money might come from

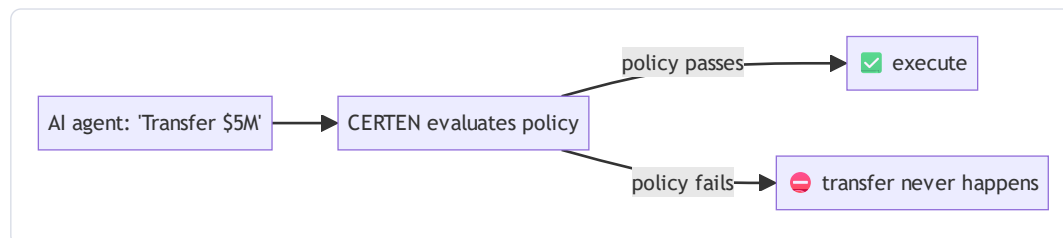
The largest value may **not** sit with end users, because approval workflows feel like infrastructure and people tend to resist paying for infrastructure directly. A few directions, roughly ordered by how promising they *seem* from my seat:



- **Path A — End users.** Small per-approval or per-account fees. My instinct is this is the weakest, for the infrastructure-pricing reason above — but you'll know better.
- **Path B — Integrations.** Another platform (exchange, custodian, wallet, lender, DAO tooling) embeds Certen and pays, while their end users may never know it's there — the shape Stripe, Plaid, Fireblocks, and Twilio took.
- **Path C — Enterprise governance.** Where large value is at stake — a sizeable DAO treasury, a bridge, a stablecoin reserve, a big device network — the question "who approves changes?" may be a much larger problem than a per-action fee implies. The framing that resonates with me: customers there may be paying **to avoid catastrophe**, not paying for approvals.

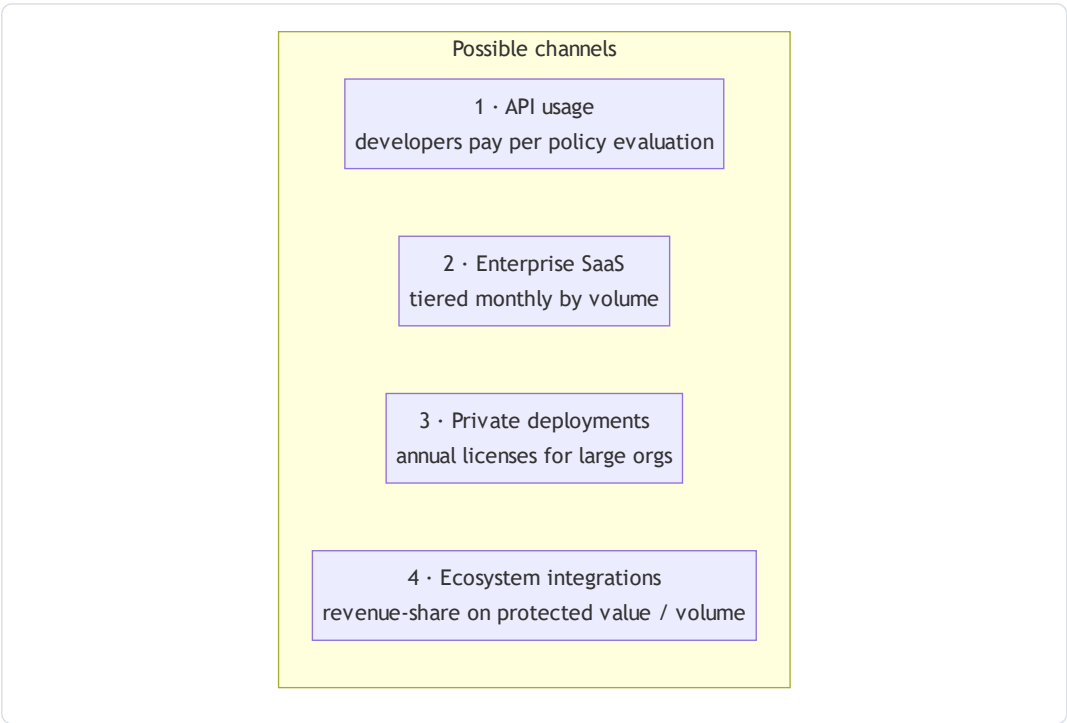
One technical angle worth surfacing

Because of the Accumulate heritage, there's a capability I think is easy to under-sell. Most conversations stop at **identity** → **authorization** → **governance** (proving *who approved*). But the engine can also sit **between intent and action and actually block it** — i.e. **identity** → **policy** → **execution**:



That's the difference between an **audit system** (records what happened) and an **enforcement system** (prevents what shouldn't). I suspect the latter is worth considerably more.

4. Some monetization shapes to consider



#	Possible channel	Who might pay	Illustrative only
1	API usage	developers	a small fee per policy evaluation
2	Enterprise SaaS	organizations	tiered monthly pricing by volume
3	Private deployments	large organizations	annual licenses
4	Ecosystem integrations	bridges, lenders, custodians	a share of protected value or volume

If any of these has legs, channel 4 is the one I find most intriguing technically: the business could scale with the **value it protects** rather than seat counts.

5. One thing I'd gently flag to avoid

From watching the space, **token-gated approvals** ("users must buy a token to approve a decision") have tripped up a lot of projects — they tend to add friction to the exact workflow you want adopted. I'd lean against it on engineering and UX grounds.

6. If it's useful: a short way to talk about the engine

One-liners to describe Certen:

- *"Execution is conditional on proof."* — the essence in five words.
- *"It decides whether an action should be allowed, and proves the answer."*
- *"Cryptographic change control for digital assets and autonomous systems."* — if a concrete, narrow framing helps.

← [Back to index](#) · You've reached the end of the original guide — continue to page 8 for the API Gateway.

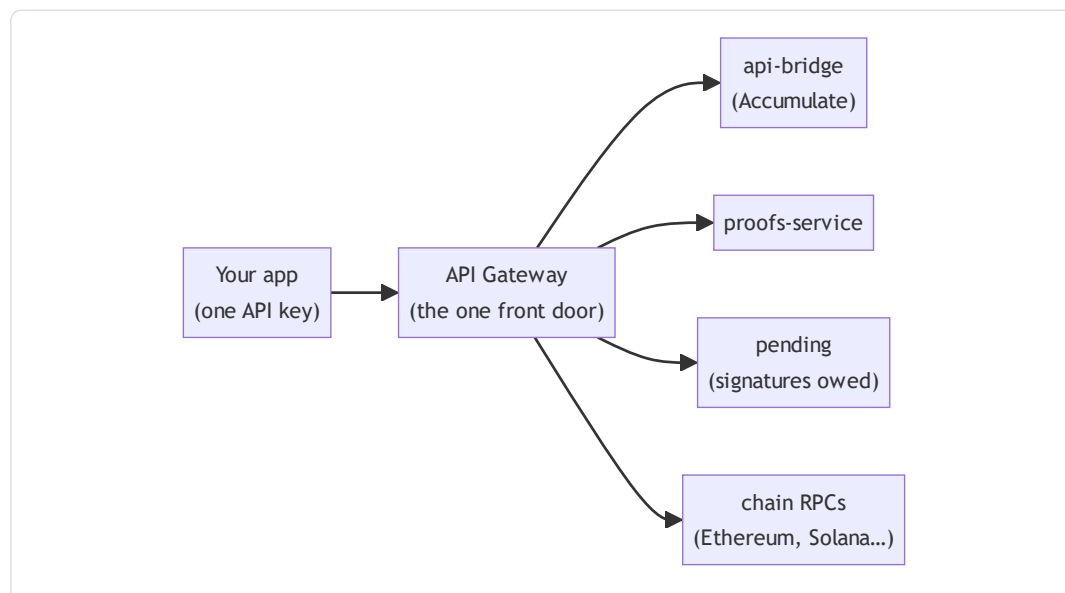
08 — The API Gateway (For Builders)

[← Back to index](#)

Who this page is for: anyone who wants to use Certen **from their own software** — a backend, a script, a custom dashboard — instead of clicking through the Certen web app and the KeyVault browser extension.

1. The one-paragraph version

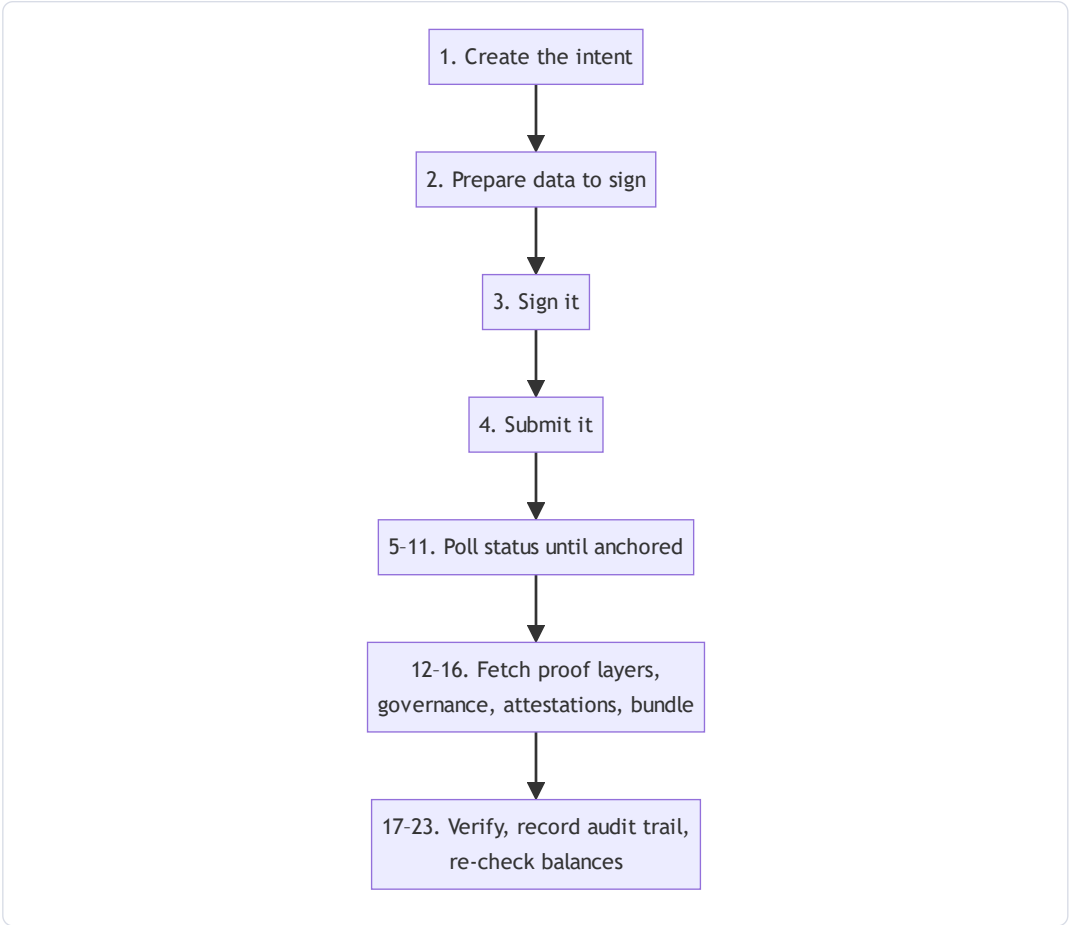
Certen is really a **team of specialist services** working together — one that talks to the Accumulate "governance brain," one that builds the cryptographic proofs, one that watches for signatures that are still needed, one that talks to each outside blockchain, and so on. That's great for power, but it means a single real-world action ("send this, prove it, and record it") can take **dozens of separate calls** across **five or six services**. The **API Gateway** is a service that exposes a **handful of simple, high-level commands**, that allows you to send one request with one API key that executes 12-or-23-step actions behind the scenes. The gateway is meant to provide users a **"Stripe-style" front door** to Certen.



2. Why this exists (instead of calling each service yourself)

You *could* talk to each Certen service directly. The web app does exactly that. But everything that makes the platform powerful — multiple chains, multi-signer governance, layered proofs — also makes the raw wiring **complicated and easy to get wrong**.

Here is a single, ordinary cross-chain transaction, done the **hard way** (this is what the web app juggles internally):



That's 23 steps across four services for one action. Do it by hand and you own every retry, every "not ready yet, try again," every error format, and every audit-log entry.

The gateway collapses that into **one call** — and adds the things a real integration needs:

What you'd build yourself	What the gateway gives you for free
Glue code across 5–6 services	One REST API, one base URL
Juggling several logins/secrets	One API key (<code>X-API-Key</code> header)
"Is it anchored yet?" polling loops	The gateway polls; you get a webhook when it's done
Your own records of who did what	A built-in audit log in the gateway's own database
Safe retries	Idempotency keys — sending the same request twice is safe
Worrying about breaking the web app	Total isolation — the gateway never touches the web app's data

3. The compound methods (the heart of the gateway)

A **compound method** is one gateway command that bundles many lower-level service calls into a single, meaningful business action. There are **seven**, and they cover the full lifecycle of using Certen.

#	Command	In plain English	What it bundles
1	<code>POST /v1/identity</code>	"Set me up." Create an identity, link blockchains, set who can approve.	~12 Accumulate calls + chain setup
2	<code>POST /v1/transaction</code>	"Do it and prove it." Run a cross-chain action and produce a proof.	The full 23-step workflow
3	<code>POST /v1/governance</code>	"Change the rules." Add/remove approvers, change thresholds.	~8 calls + signing
4	<code>GET /v1/proof/{id}</code>	"Show me the receipt." Get the complete proof with all its parts.	~6 proof-service calls
5	<code>GET /v1/inbox</code>	"What needs my signature?" List actions waiting on approval.	Pending-discovery + enrichment
6	<code>POST /v1/inbox/{id}/sign</code>	"I approve." Sign/vote on a pending action.	Signing-path lookup + submit
7	<code>GET /v1/portfolio</code>	"How am I doing?" All balances + identity status, every chain at once.	15+ chain queries + Accumulate

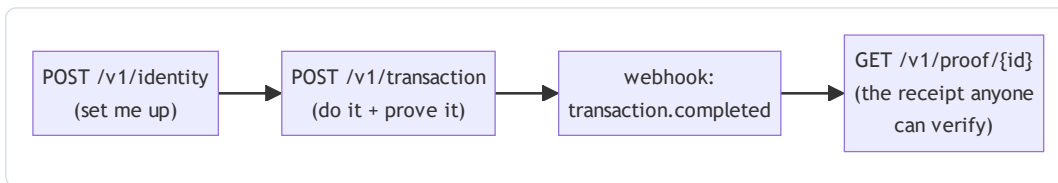
Two of these are the real **end-to-end "composite" commands** — the ones that turn a mountain of work into a single request. They're worth calling out:

★ `POST /v1/identity` — onboard in one call

Behind this single request, the gateway creates your on-chain identity (your "ADI"), sets up its key book and approval rules, funds it with the network "credits" it needs to operate, **waits for each step to actually settle** (the network is eventually-consistent, so this is fiddly to do by hand), links every blockchain you asked for, and registers you so the gateway starts watching for work on your behalf. You get back a ready-to-use identity.

★ `POST /v1/transaction` — act + prove in one call

This is the flagship. One request kicks off the entire "do the action on chain A, anchor the decision, generate the layered proof, confirm it" pipeline. You can either **wait for the webhook** that says `transaction.completed` (with the full proof attached), or poll for status. Either way, *you* never orchestrate the 23 steps — the gateway does.

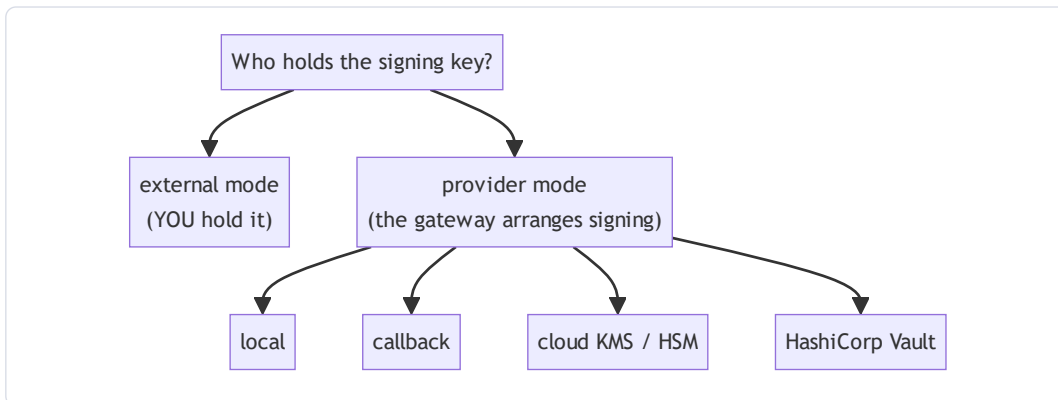


That four-box picture is the **entire Certen experience** for a builder. Almost everything else is a variation on it.

4. Keys and signing — your options

Every action on Accumulate has to be **signed** by a private key. This is the question most builders care about most, so here it is plainly.

The gateway gives you **two broad styles**, chosen per identity:



Style A — `external` (you keep the keys)

This is the default. You never hand Certen a private key. When an action needs a signature, the gateway **prepares** the exact bytes to sign and returns them; you sign with **whatever tool you like** — the Accumulate CLI, the Certen KeyVault browser extension, or any standard Ed25519 signer — and then **POST the signature** back. The gateway submits it. Maximum control; keys never leave you.

Style B — `provider` (the gateway arranges the signing)

If you'd rather not run a signer yourself, you tell the gateway which **signing provider** to use. There are **six**, and the choice is entirely yours:

Provider	Where the key lives	Best for	Needs a cloud account?
local	Generated by the gateway, encrypted at rest in its own database	Getting started fast, dev/test, self-hosting	No
callback	On your own server — the gateway asks you to sign	"Bring your own signer"; keys never touch Certen	No
aws-kms	Amazon's Key Management Service	Teams already on AWS	Yes (AWS)
azure-keyvault	Microsoft Azure Key Vault	Teams already on Azure	Yes (Azure)
gcp-kms	Google Cloud KMS	Teams already on Google Cloud	Yes (Google)
hashicorp-vault	HashiCorp Vault (Transit engine)	Enterprises with their own vault	No — Vault can be self-hosted

A note on the `local` provider and seed phrases. When you choose `local`, the gateway creates an Ed25519 key for you and stores the secret **encrypted** (AES-256-GCM, with a per-identity key derived from a master secret only the operator holds). If you ask it to generate a **BIP-39 mnemonic** (the familiar 12/24-word backup phrase), the gateway will **not** put that phrase in the normal response — instead it hands you a **one-time, short-lived link** to fetch it exactly once. After that, it's gone. This keeps the phrase out of logs and proxies.

5. How to set up and use each option

You choose your signing style **when you create the identity**, in the same `POST /v1/identity` call. Below are the shapes — deliberately minimal.

External (you keep the keys): just give your public-key hash. You'll sign later, yourself.

```
{
  "name": "acme-corp",
  "signing_mode": "external",
  "public_key_hash": "a1b2c3...",
  "chains": ["ethereum-sepolia", "solana-devnet"]
}
```

Provider — `local` (gateway-managed, optional seed phrase):

```
{
  "name": "acme-corp",
  "signing_mode": "provider",
  "signing_provider": {
    "type": "local",
    "config": { "method": "mnemonic", "mnemonic_strength": 256 }
  }
}
```

The response includes a one-time `mnemonic_retrieval` link if you asked for a phrase — **fetch it once and store it safely**.

Provider — `callback` (bring your own signer): point the gateway at an HTTPS endpoint you control. When something needs signing, the gateway sends the hash; your endpoint returns the signature. Add an `hmac_secret` so each side can verify the other.

```
{
  "name": "acme-corp",
  "signing_mode": "provider",
  "signing_provider": {
    "type": "callback",
    "config": {
      "signing_url": "https://acme.com/certen/sign",
      "public_key": "d4e5f6...",
      "hmac_secret": "a-shared-secret-at-least-16-chars"
    }
  }
}
```

Provider — cloud KMS or Vault: name the key you already manage. The gateway calls your cloud/vault to sign; **the private key never leaves it.**

```
{
  "name": "acme-corp",
  "signing_mode": "provider",
  "signing_provider": {
    "type": "aws-kms",
    "config": { "key_id": "arn:aws:kms:...", "region": "us-east-1" }
  }
}
```

Operator footnote (for whoever runs the gateway, not the API caller): the `local` provider needs a `KEY_ENCRYPTION_MASTER_SECRET` set in the gateway's environment; mnemonic delivery needs a `MNEMONIC_SIGNING_SECRET`; the cloud providers need that cloud's normal credentials. None of this is the API caller's concern — they just pick a `type`.

After setup, **using** the key is automatic: in `provider` mode the gateway signs for you whenever a method needs it; in `external` mode it returns the bytes and waits for your signature. Same seven commands either way.

6. Recap

- The gateway is the **one simple front door** to a platform that is, underneath, a team of specialist services.
- It exists so you make **one call** instead of orchestrating **dozens** — and it throws in one API key, webhooks, idempotency, and an audit log.
- The **seven compound methods** cover the whole lifecycle; **create identity** and **run transaction** are the big end-to-end composites that hide the most work.
- For keys you have **two styles** — keep them yourself (`external`) or let the gateway arrange signing (`provider`) — and **six providers**, of which **only three involve a cloud KMS**. A purely **local** or **bring-your-own** setup needs no cloud at all.

